



COMSATS University Islamabad, Sahiwal Campus

MediBot

**A project presented to
COMSATS Institute of Information Technology, Islamabad Sahiwal
Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

Immad Ikram

CIIT/FA22-BCS-036/SWL

Muhammad Nabeel

CIIT/FA22-BCS-077/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Immad Ikram

Muhammad Nabeel

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “**MediBot**” was developed by **Immad Ikram (CIIT/FA22-BCS-036/SWL)** and **Muhammad Nabeel (CIIT/FA22-BCS-077/SWL)** under the supervision of “**Mr. Ali Sher Kashif**” and co supervisor “**Dr. Fakhar Mustafa**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Mr. Ali Sher Kashif

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Imran Sarwar Bajwa

Professor

Department of Computer Science

Islamia University Bahawalpur

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

MediBot is a web-based telemedicine platform developed to simplify healthcare consultation, appointment scheduling, and doctor–patient interaction through a secure and intelligent digital environment. Designed with a focus on accessibility, automation, and usability, the platform bridges the gap between patients and verified medical professionals by providing a centralized system for managing online medical consultations. The system enables patients to register, search for doctors based on specialization, view profiles, purchase credits, book appointments, and communicate with doctors through real-time video consultations.

For doctors, MediBot provides a dedicated dashboard to manage availability, view scheduled appointments, conduct video consultations, and monitor earnings. Doctors can set available time slots, update professional information, and handle consultation requests efficiently. Administrators are provided with a comprehensive control panel to verify doctor credentials, manage users, monitor consultations, and process payout requests. This role-based system ensures smooth communication between patients, doctors, and administrators while maintaining operational transparency and data consistency.

A key feature of MediBot is its structured credit-based appointment model, which allows patients to purchase credits and use them to book consultations. This approach simplifies payment handling and ensures transparency in transactions. The platform also integrates secure real-time video consultation functionality, enabling remote healthcare access without the need for third-party communication tools. Authentication and access control are handled using secure role-based login mechanisms, ensuring that sensitive patient and doctor information remains protected.

The system follows a modular architecture that promotes scalability, maintainability, and efficient development. The user interface, built using React 19, Next.js 15, and Tailwind CSS, focuses on simplicity, responsiveness, and intuitive navigation. The backend, powered by Prisma ORM and PostgreSQL, ensures secure data storage and efficient handling of appointments, user profiles, and credit transactions. Integration of Clerk authentication and Vonage Video API enhances system security and communication reliability.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor of **Mr. Ali Sher Kashif** and co-supervisor **Dr. Fakhar Mustafa**. Without their personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are deeply thankful for their encouragement and continual support during this work.

We also extend our heartfelt gratitude to our families and friends who have been a constant source of support and motivation throughout this journey.

Immad Ikram

Muhammad Nabeel

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
NoSQL	Non-Structured Query Language
REST	Representational State Transfer
UI	User Interface

Table of Contents

1 Introduction.....	1
1.1 Relevance to Course Modules	1
1.2 Project Background	3
1.3 Literature Review	3
1.4 Analysis from Literature Review	4
1.5 Methodology and Software Lifecycle	5
1.5.1 Rationale behind Selected Methodology:	6
2 Problem Definition.....	8
2.1 Problem Statement.....	8
2.2 Deliverables and Development Requirements.....	9
2.3 Current System	10
3 Requirement Analysis.....	13
3.1 Use Case Diagram	13
3.2 Use Case Description.....	14
3.3 Functional Requirements.....	26
3.3.1 Patient Login and Signup	26
3.3.2 Patient Payment (Credits Purchase).....	26
3.3.3 Patient Find Doctor	26
3.3.4 Patient Book Appointment	27
3.3.5 Patient Video Call with Doctor.....	27
3.3.6 Doctor Login and Signup.....	27
3.3.7 Doctor Dashboard.....	27
3.3.8 Doctor Request Payout	28
3.3.9 Doctor Set Availability	28
3.3.10 Admin Verify and Manage Doctors.....	28
3.3.11 Admin Payout Management	28
3.4 Non-Functional Requirements.....	28
4 Design and Architecture.....	33
4.1 System Architecture	33
4.3 Process Flow/Representation.....	37
4.3.1 Activity Diagrams.....	37
4.4 Design Models.....	42
4.4.1 Sequence Diagram.....	42
4.4.2 Class Diagram.....	44
5 Implementation	48
5.1 Algorithm.....	48
5.2 External APIs.....	57
5.3 User Interface	58
6 Testing and Evaluation.....	65
6.1 Manual Testing	65

6.1.1 System Testing	66
6.1.2 Unit Testing	67
6.1.3 Functional Testing	68
6.1.4 Integration Testing.....	69
6.2 Tools and Technologies.....	70
6.3 Summary.....	71
7 Conclusion.....	73
7.1 Future Work.....	74
8 References.....	76

List of Figures

FIGURE 3. 1: MEDIBOT USE CASE.....	13
FIGURE 4. 1: MEDIBOT SYSTEM ARCHITECTURE.....	35
FIGURE 4. 2: MEDIBOT ER DIAGRAM.....	36
FIGURE 4. 3: PATIENT: BUYING CREDITS & BOOKING APPOINTMENT ACTIVITY DIAGRAM.....	39
FIGURE 4. 4: DOCTOR: MANAGING AVAILABILITY & VIDEO ACTIVITY DIAGRAM.....	40
FIGURE 4. 5: ADMIN: VERIFYING DOCTORS & APPROVING PAYOUTS ACTIVITY DIAGRAM.....	41
FIGURE 4. 6: SEQUENCE DIAGRAM FOR PATIENT CORE PROCESS	42
FIGURE 4. 7: SEQUENCE DIAGRAM FOR DOCTOR CORE PROCESS	43
FIGURE 4. 8: SEQUENCE DIAGRAM FOR ADMIN CORE PROCESS.....	44
FIGURE 4. 9: CLASS DIAGRAM	45
FIGURE 5. 1: SIGN-IN SCREEN	59
FIGURE 5. 2: SIGN-UP SCREEN	60
FIGURE 5. 3: DOCTOR PROFILE AND APPOINTMENT BOOKING SCREEN.....	61
FIGURE 5. 4: DOCTOR DASHBOARD SCREEN	62
FIGURE 5. 5: ADMIN MANAGE DOCTORS SCREEN.....	63

LIST OF TABLES

TABLE 1. 1: RELEVANCE TO COURSE MODULES	2
TABLE 3. 1: USER REGISTRATION AND LOGIN	14
TABLE 3. 2: PATIENT BUY CREDITS / PAYMENT.....	15
TABLE 3. 3: PATIENT FIND DOCTOR.....	16
TABLE 3. 4: PATIENT BOOK APPOINTMENT.....	17
TABLE 3. 5: PATIENT VIDEO CALL WITH DOCTOR	18
TABLE 3. 6: DOCTOR CHECK DASHBOARD	19
TABLE 3. 7: DOCTOR REQUEST PAYOUT.....	20
TABLE 3. 8: DOCTOR VIEW APPOINTMENTS.....	21
TABLE 3. 9: DOCTOR SET AVAILABILITY	22
TABLE 3. 10: ADMIN VERIFY DOCTORS	23
TABLE 3. 11: ADMIN MANAGE DOCTORS	24
TABLE 3. 12: ADMIN MANAGE PAYOUTS.....	25
TABLE 5. 1: EXTERNAL APIS	57
TABLE 6. 1: SYSTEM TESTING TABLE	66
TABLE 6. 2: UNIT FUNCTION TABLE	67
TABLE 6. 3: TEST CASE TABLE	68
TABLE 6. 4: INTEGRATION TESTING TABLE.....	69
TABLE 6. 5: TOOLS AND TECHNOLOGIES TABLE	70

Chapter 1

INTRODUCTION

1 Introduction

MediBot is a Software-as-a-Service (SaaS) application developed as a Final Year Project at COMSATS University Islamabad, Sahiwal Campus. It is a modern, AI-assisted telemedicine platform designed to bridge the communication gap between patients and verified medical professionals by providing a secure and user-friendly digital healthcare environment. The system enables patients to easily search for specialists across multiple medical fields, view doctor profiles, check availability, and book appointments without the need for physical visits. Patients can then communicate with doctors through secure, real-time video consultations, ensuring accessibility, convenience, and time efficiency.

On the other hand, doctors are provided with a dedicated dashboard where they can manage their availability, update professional information, verify their credentials, and handle consultation requests. The platform also allows doctors to track completed consultations and receive payouts accordingly. MediBot operates on a credit-based model, where patients purchase credit packages and use those credits to book appointments with doctors. This approach simplifies payment handling, ensures transparency, and provides a flexible billing mechanism for both patients and healthcare providers.

The platform is developed using modern web technologies including React 19, Next.js 15, Tailwind CSS, Prisma ORM, PostgreSQL database, Clerk Authentication, Vonage Video API, and Shadcn UI. These technologies ensure scalability, performance, and a responsive user experience across devices. MediBot is structured around three major role-based panels: Patient Panel, Doctor Panel, and Admin Panel. Each panel offers a comprehensive set of features designed to efficiently manage telemedicine workflows, including appointment management, user verification, credit handling, analytics, and system monitoring. This architecture ensures smooth coordination between all stakeholders while maintaining security, reliability, and ease of use.

1.1 Relevance to Course Modules

The development of the **MediBot** platform integrates multiple concepts, theories, and technical skills acquired throughout the **Bachelor of Science in Computer Science (BSCS)** degree program. This project combines areas such as software engineering, artificial intelligence, database management, web development, and human-computer interaction to create a comprehensive and intelligent real estate management system.

Each course module contributes to different aspects of the system as outlined below:

Table 1. 1: Relevance to Course Modules

Course Module	Application in StayBnB
Software Engineering	Software Engineering SRS & SDD documents, UML diagrams
Database Systems	Designed and implemented data storage and retrieval mechanisms using MongoDB for user, property, and transaction data.
Web Technologies	Developed responsive web interfaces using React and Tailwind CSS, ensuring seamless interaction and usability.
Artificial Intelligence	Incorporated AI and ML models for property recommendations, market analysis, and price forecasting.
Human-Computer Interaction (HCI)	Applied UI/UX design principles to enhance user engagement and accessibility.
Information Security	Implemented JWT-based authentication, secure data transmission, and rate limiting to prevent unauthorized access.

Table 1.1.1: Relevance to Course Module outlines the project not only reinforces theoretical understanding but also demonstrates practical implementation of interdisciplinary knowledge, aligning with the academic objectives of the Computer Science curriculum.

By integrating these subjects, the project provides a comprehensive demonstration of real-world application of the theoretical learning gained over eight semesters.

1.2 Project Background

Access to timely and reliable healthcare has become a major challenge worldwide, particularly in regions where patients face long waiting times, limited access to specialists, and difficulty in navigating healthcare services. Numerous studies and reports highlight that a significant portion of the population struggles with delayed medical consultations, lack of awareness about available doctors, and poor appointment management systems. According to recent healthcare technology research, digital platforms and telemedicine solutions have shown strong potential in reducing patient burden by simplifying appointment booking, improving doctor-patient communication, and enabling remote care.

Several existing platforms, such as basic hospital management systems and general teleconsultation apps, provide partial solutions but still suffer from issues like complicated interfaces, lack of integrated patient profiles, limited availability data, and poor user experience. Many applications also fail to offer a unified platform where users can discover doctors, view their profiles, check real-time availability, and book appointments seamlessly. These gaps indicate a clear need for a more intuitive and accessible healthcare assistance system.

MediBot was developed in response to these identified challenges. Inspired by the literature emphasizing the importance of digital health accessibility, the project aims to offer a streamlined solution where users can easily find doctors, explore their professional details, and book appointments anytime, anywhere. By addressing the shortcomings of existing platform particularly the lack of integrated, user-centered design MediBot provides a modern, efficient, and reliable healthcare experience. The project focuses on enhancing convenience, improving patient engagement, and supporting informed decision-making through well-structured information and simple navigation.

1.3 Literature Review

The integration of artificial intelligence in telemedicine and digital healthcare platforms has grown significantly in recent years. Existing telemedicine systems such as Practo, Teladoc, and Doctor on Demand provide online appointment booking, doctor listings, and video consultations; however, many of these platforms offer limited intelligent automation and

personalization. Most systems focus primarily on communication between patients and doctors, with minimal use of AI for smart recommendations, patient triage, or workflow optimization.

Research studies highlight the growing importance of AI in telemedicine environments. The study “Artificial Intelligence in Telemedicine and Remote Healthcare” (Sharma & Gupta, 2021) emphasizes how AI can assist in symptom analysis, patient prioritization, and improved healthcare accessibility. Another research work, “AI-Based Medical Appointment Recommendation System” (Khan et al., 2022), demonstrates the effectiveness of machine learning algorithms in recommending suitable doctors based on patient history, specialization, and availability. These studies confirm that AI-driven telemedicine platforms can significantly enhance user experience and reduce administrative overhead.

Despite these advancements, many localized healthcare systems still rely on manual appointment booking and basic communication features. Existing platforms often lack integrated credit-based payment handling, automated doctor verification workflows, and intelligent scheduling systems. Moreover, real-time video consultation integration and secure authentication mechanisms are not fully optimized in many current solutions, resulting in inefficiencies and fragmented user experiences

1.4 Analysis from Literature Review

The literature review demonstrates that incorporating artificial intelligence into telemedicine platforms significantly improves system efficiency, accessibility, and user satisfaction. While existing systems provide essential functionalities such as appointment booking and virtual consultations, most solutions remain limited in automation, personalization, and workflow management. Some platforms focus solely on doctor listings, while others lack intelligent scheduling, secure credit handling, and administrative monitoring tools.

This analysis highlights the need for a unified telemedicine system that integrates AI-assisted decision-making, secure communication, and role-based workflow management. Such a system would enhance operational efficiency for healthcare providers while improving accessibility and convenience for patients.

1.4.1 Key Findings

The review identifies a technological gap between traditional telemedicine systems and modern AI-assisted healthcare platforms. Current trends show that users expect

personalized doctor recommendations, streamlined appointment booking, and secure real-time consultations. Additionally, scalable web technologies enable seamless integration of authentication, payment models, and communication services within a single platform. Systems that combine role-based dashboards, AI-assisted features, and real-time communication tools tend to achieve better usability and system performance. Furthermore, credit-based payment models provide transparency and simplify financial transactions between patients and healthcare providers.

1.4.2 Conceptualization of StayBnB

Based on these findings, MediBot was conceptualized as a complete AI-assisted telemedicine ecosystem. Instead of functioning as a basic appointment booking system, MediBot integrates intelligent scheduling, secure video consultation, doctor verification workflows, and credit-based payments. The platform provides three role-based panels — Patient, Doctor, and Admin — each designed to handle specific telemedicine operations efficiently. This approach ensures improved communication, better resource management, and a more reliable healthcare experience. MediBot aims to deliver a modern, scalable, and user-friendly solution that addresses the limitations identified in existing telemedicine platforms.

1.5 Methodology and Software Lifecycle

The Agile Software Development Lifecycle (SDLC) was adopted for the MediBot project to ensure flexibility, iterative development, and continuous feedback. Agile was selected because telemedicine platforms require frequent updates, testing, and integration of multiple services such as authentication, video conferencing, and payment systems. The iterative nature of Agile allows gradual development of features while continuously improving system usability and performance.

Key Phases:

1. Requirement Gathering:

Identification of functional and non-functional requirements, including patient registration, doctor verification, appointment booking, credit-based payments, and secure video consultation features.

2. System Design:

Creation of system architecture, database schema, UML diagrams, and user interface wireframes. Role-based panels for Patient, Doctor, and Admin were designed to manage telemedicine workflows.

3. Implementation:

Development of the frontend using React 19, Next.js 15, and Tailwind CSS, while backend logic and database integration were implemented using Prisma ORM and PostgreSQL.

4. Integration of AI Services:

Integration of Clerk Authentication for secure login, Vonage Video API for real-time video consultations, and credit-based payment workflow for appointment handling.

5. Testing and Validation:

Unit testing, integration testing, and system testing were performed to ensure functionality, security, and performance across all modules.

6. Deployment:

Deployment of the web application on cloud platforms to ensure accessibility, scalability, and availability for both patients and doctors.

7. Maintenance:

Continuous monitoring, feature enhancement, bug fixing, and performance optimization to maintain long-term system reliability.

1.5.1 Rationale behind Selected Methodology:

The Agile methodology was selected due to its iterative and incremental development approach, which supports continuous improvement and flexible requirement handling. Unlike traditional waterfall models, Agile allows parallel development of independent modules such as authentication, appointment management, video consultation, and credit handling. This flexibility is particularly important for MediBot, where multiple components must be integrated seamlessly. Agile also encourages regular testing, stakeholder feedback, and adaptive planning, ensuring that the final system remains scalable, secure, and aligned with telemedicine requirements.

Chapter 2

PROBLEM DEFINITION

2 Problem Definition

The healthcare sector is rapidly evolving, with an increasing demand for accessible, efficient, and reliable telemedicine solutions. However, despite advancements in digital healthcare, many existing telemedicine platforms still lack intelligent automation, seamless communication, and efficient workflow management. Most systems provide only basic appointment booking and communication features without incorporating smart scheduling, secure payment handling, or comprehensive administrative control. This often results in delayed consultations, poor user experience, and inefficient coordination between patients and healthcare providers.

The MediBot project was initiated to address these challenges by introducing an AI-assisted telemedicine web application that intelligently connects patients with verified doctors. The system focuses on secure video consultations, credit-based appointment booking, and role-based dashboards for efficient healthcare management. This chapter defines the problem in detail, outlines the system deliverables, and discusses the limitations of existing telemedicine systems that MediBot aims to improve.

2.1 Problem Statement

Traditional telemedicine platforms are mostly limited to doctor listings and manual appointment scheduling. These systems often lack intelligent scheduling, automated verification, and structured payment mechanisms. As a result, patients face difficulties in finding suitable doctors based on specialization and availability, while healthcare providers struggle to manage consultation requests and schedules efficiently.

A deeper analysis highlights several critical issues. Current systems do not provide smart appointment handling or streamlined credit-based payment models. Many platforms lack secure, integrated video consultation features, forcing users to rely on third-party tools. The user experience is often limited due to static interfaces and minimal personalization. Administrators also face challenges in monitoring doctor verification, managing users, and tracking platform activity.

Furthermore, existing systems often lack role-based architecture, resulting in inefficient data handling and poor workflow separation between patients, doctors, and administrators. Security concerns, limited scalability, and inadequate authentication mechanisms also contribute to operational inefficiencies. These limitations create a strong need for a modern telemedicine

solution that integrates secure communication, automated workflows, and scalable architecture.

MediBot addresses these challenges by providing a centralized, secure, and intelligent telemedicine platform. The system enhances appointment booking, enables real-time video consultations, manages doctor verification, and implements a credit-based consultation model, ensuring a smooth and reliable healthcare experience

2.2 Deliverables and Development Requirements

The MediBot project aims to deliver a complete, fully functional telemedicine web application with role-based access, secure communication, and credit-based consultation management. The expected deliverables and development components are listed below:

2.2.1 Major Deliverables:

2.2.1.1 AI-Assisted Telemedicine Web Application – enabling patients to search doctors, book appointments, and conduct video consultations.

2.2.1.2 Patient Panel – allowing users to manage profiles, purchase credits, book consultations, and view appointment history.

2.2.1.3 Doctor Panel – enabling doctors to manage schedules, verify credentials, handle appointments, and track earnings.

2.2.1.4 Admin Dashboard – for managing users, verifying doctors, monitoring consultations, and controlling platform operations.

2.2.1.5 Secure Authentication System – implementing role-based login and access control using Clerk authentication.

2.2.1.6 Video Consultation Integration – real-time communication using Vonage Video API.

2.2.1.7 Credit-Based Payment System – allowing patients to purchase credits and use them for booking consultations.

2.2.1.8 Deployment Setup – cloud-based deployment for scalability and accessibility.

2.2.1.9 Technical Documentation and Source Code Repository – ensuring maintainability and development transparency.

2.2.2 Development Requirements:

2.2.2.1 Frontend: Next.js, TailwindCSS, Framer Motion

2.2.2.2 Backend: Node.js, Express.js

2.2.2.3 Database: NeonDB

2.2.2.4 Video Integration: Vonage Video API

2.2.2.5 Chatbot Integration: Namiru Ai

2.2.2.5 Authentication: Clerk Authentication (role-based access)

2.2.2.6 Deployment: Vercel (Frontend)

2.2.2.7 Tools: Git, Visual Studio Code, Postman

Together, these deliverables ensure that StayBnB functions as a scalable, secure, and intelligent property platform with seamless integration between AI and full-stack components.

2.3 Current System

In the current scenario, many telemedicine platforms provide limited functionality, primarily focusing on doctor listings and simple appointment booking. These systems often rely on manual scheduling and lack real-time communication integration. Patients typically receive generic search results without intelligent filtering based on specialization, availability, or consultation type.

Additionally, most existing systems do not include a structured admin panel for managing doctor verification and monitoring platform activity. This lack of administrative control leads to inefficiencies in maintaining data accuracy and system reliability. Many platforms also lack secure authentication mechanisms and proper role-based access control, which can compromise data privacy and user management.

Another limitation of current systems is the absence of a structured payment model. Users often face difficulties in handling consultation payments, while doctors lack a transparent system for tracking earnings. Furthermore, some platforms depend on external communication tools instead of integrated video consultation features, reducing usability and increasing complexity.

MediBot overcomes these limitations by providing a fully integrated telemedicine architecture. The system offers role-based dashboards, secure authentication, credit-based booking, real-

time video consultations, and efficient administrative control. By combining modern web technologies with scalable cloud deployment, MediBot delivers a reliable and user-friendly healthcare platform that improves communication between patients and medical professionals while enhancing overall system efficiency.

Chapter 3

REQUIREMENT ANALYSIS

3 Requirement Analysis

Requirement Analysis, or Requirement Engineering, is the process of defining user expectations for new software to be built or modified. In software engineering, it is sometimes referred to loosely by names such as requirements gathering or requirements capturing. It involves gathering, analyzing, documenting, validating, and managing the needs and conditions required to meet stakeholder expectations, ensuring alignment with project goals.

3.1 Use Case Diagram

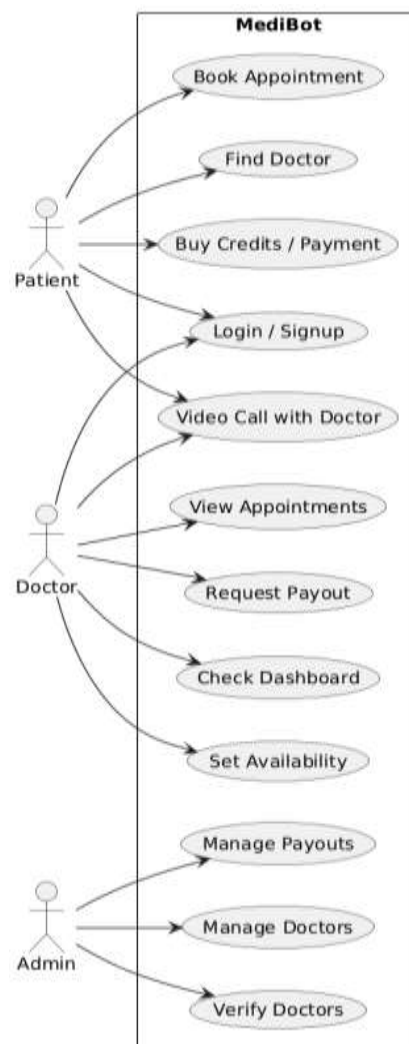


Figure 3. 1: MediBot Use Case

3.2 Use Case Description

Table 3. 1: User Registration and login

Use Case ID	UC#1
Use Case Name	Login / Signup
Description	Allows users (patients or doctors) to register or log in to the MediBot system.
Primary Actor	Patient / Doctor
Secondary Actor	Admin (for doctor verification)
Pre-Condition	User must have internet access and a valid email ID.
Post-Condition	User is authenticated and granted access to their respective dashboard.
Basic Flow	1. User navigates to login/signup page.2. User enters credentials (email/password) or signs up.3. System validates credentials.4. User is redirected to the dashboard.
Alternate Flow	1a. User enters incorrect credentials → System displays an error.1b. User forgets password → System sends password reset link.

Table 3. 2: Patient Buy Credits / Payment

Use Case ID	UC#2
Use Case Name	Buy Credits / Payment
Description	Allows patients to purchase credits to book appointments or use video call services.
Primary Actor	Patient
Secondary Actor	Payment Gateway
Pre-Condition	Patient must be logged in.
Post-Condition	Credits are added to the patient account; payment transaction is logged.
Basic Flow	1. Patient navigates to the payment section. 2. Patient selects number of credits to buy. 3. System redirects to payment gateway. 4. Payment is processed .5. Credits are added to the patient account.
Alternate Flow	a. Payment fails → System displays error and prompts retry. b. Patient cancels payment → Credits are not added.

Table 3. 3: Patient Find Doctor

Use Case ID	UC#3
Use Case Name	Find Doctor
Description	Allows patients to search for doctors by specialization, availability, or location.
Primary Actor	Patient
Secondary Actor	None
Pre-Condition	Patient must be logged in and have an active internet connection.
Post-Condition	Patient sees a list of available doctors matching search criteria.
Basic Flow	1. Patient enters search criteria. 2. System queries doctor database. 3. System displays a list of doctors. 4. Patient selects a doctor for appointment.
Alternate Flow	a. No doctors found → System displays “No doctors available” message. b. Invalid search input → System prompts correction.

Table 3. 4: Patient Book Appointment

Use Case ID	UC#4
Use Case Name	Book Appointment
Description	Allows patients to schedule an appointment with a doctor based on available time slots.
Primary Actor	Patient
Secondary Actor	Doctor
Pre-Condition	Patient must be logged in and have sufficient credits.
Post-Condition	Appointment is confirmed and added to both patient and doctor schedules.
Basic Flow	1. Patient selects a doctor. 2. Patient chooses an available time slot. 3. System verifies credit balance. 4. Appointment is booked. 5. Notifications sent to patient and doctor.
Alternate Flow	a. Insufficient credits → System prompts purchase. b. Selected time slot unavailable → System prompts selection of a new slot.

Table 3. 5: Patient Video Call with Doctor

Use Case ID	UC#5
Use Case Name	Video Call with Doctor
Description	Enables real-time video consultation between patient and doctor.
Primary Actor	Patient / Doctor
Secondary Actor	Vonage Video API
Pre-Condition	Both patient and doctor must be logged in, and an appointment must be scheduled.
Post-Condition	Video session completed; call logs are recorded in the system.
Basic Flow	1. Patient clicks “Join Video Call”. 2. System initiates call via Vonage API. 3. Doctor joins call. 4. Video consultation occurs. 5. Call ends and logs are saved.
Alternate Flow	a. Doctor unavailable → System notifies patient and allows rescheduling. b. Network issues → System attempts reconnect or ends call gracefully.

Table 3. 6: Doctor Check Dashboard

Use Case ID	UC#6
Use Case Name	Check Dashboard
Description	Allows doctors to view appointments, patient details, and notifications.
Primary Actor	Doctor
Secondary Actor	None
Pre-Condition	Doctor must be logged in.
Post-Condition	Doctor sees updated appointments and patient requests.
Basic Flow	1. Doctor logs in. 2. System retrieves upcoming appointments and notifications. 3. Dashboard displays the data.
Alternate Flow	a. No appointments → System shows “No appointments scheduled”.

Table 3. 7: Doctor Request Payout

Use Case ID	UC#7
Use Case Name	Request Payout
Description	Allows doctors to request payment for completed appointments.
Primary Actor	Doctor
Secondary Actor	Admin
Pre-Condition	Doctor must have completed appointments and minimum credit balance for payout.
Post-Condition	Payout request is sent to admin and processed.
Basic Flow	1. Doctor selects “Request Payout”. 2. System verifies eligible balance. 3. Request is sent to admin. 4. Admin processes payout. 5. System updates doctor’s account.
Alternate Flow	a. Insufficient balance → System shows error. b. Admin rejects request → System notifies doctor.

Table 3. 8: Doctor View Appointments

Use Case ID	UC8
Use Case Name	View Appointments
Description	Allows doctors to view scheduled appointments with patients.
Primary Actor	Doctor
Secondary Actor	None
Pre-Condition	Doctor must be logged in.
Post-Condition	Doctor sees all upcoming and past appointments.
Basic Flow	1. Doctor opens appointments page. 2. System retrieves appointments. 3. Appointments are displayed chronologically.
Alternate Flow	a. No appointments → System shows “No appointments scheduled”.

Table 3. 9: Doctor Set Availability

Use Case ID	UC#9
Use Case Name	Set Availability
Description	Allows doctors to define or update available time slots for appointments.
Primary Actor	Doctor
Secondary Actor	None
Pre-Condition	Doctor must be logged in.
Post-Condition	Availability is updated in the system; patients can book appointments in those slots.
Basic Flow	1. Doctor navigates to availability page. 2. Doctor selects time slots .3. System saves availability. 4. Patients can now see updated slots.
Alternate Flow	a. Selected time conflicts with existing appointments → System prompts to adjust.

Table 3. 10: Admin Verify Doctors

Use Case ID	UC#10
Use Case Name	Verify Doctors
Description	Admin verifies doctor credentials before enabling account access.
Primary Actor	Admin
Secondary Actor	None
Pre-Condition	Doctor must have completed signup.
Post-Condition	Verified doctors can access system; unverified remain restricted.
Basic Flow	1. Admin reviews doctor credentials. 2. Admin approves or rejects verification. 3. System updates doctor status.
Alternate Flow	a. Invalid credentials → Admin rejects verification; doctor notified.

Table 3. 11: Admin Manage Doctors

Use Case ID	UC#11
Use Case Name	Manage Doctors
Description	Admin can update, disable, or manage doctor profiles.
Primary Actor	Admin
Secondary Actor	None
Pre-Condition	Admin must be logged in.
Post-Condition	Doctor profiles updated or disabled as needed.
Basic Flow	1. Admin navigates to doctor management page. 2. Admin selects doctor profile. 3. Admin edits or disables profile .4. Changes saved in the system.
Alternate Flow	a. Invalid input → System prompts correction.

Table 3. 12: Admin Manage Payouts

Use Case ID	UC#12
Use Case Name	Manage Payouts
Description	Admin processes payout requests from doctors.
Primary Actor	Admin
Secondary Actor	Doctor
Pre-Condition	Doctors must have pending payout requests.
Post-Condition	Payouts are processed; doctor accounts updated.
Basic Flow	1. Admin opens payout requests. 2. Admin reviews requests. 3. Admin approves payouts. 4. System updates doctor accounts and logs transactions.
Alternate Flow	a. Invalid request → System notifies admin and rejects payout.

3.3 Functional Requirements

Functional requirements define the specific behaviors, operations, and services that the **MediBot Platform** must perform to fulfill its intended objectives. These requirements describe how the system should behave under specific conditions and how users and administrators will interact with the platform to accomplish their tasks. The following subsections outline the major functionalities of the StayBnB system.

3.3.1 Patient Login and Signup

The system shall allow patients to create an account and log in to access MediBot. When a patient opens the application, the system displays a login and signup interface. The patient can register by providing name, email, password, and phone number. The system validates the email format and password strength before creating the account. Registered patients can log in using their credentials and are redirected to the patient dashboard upon successful authentication. If invalid credentials are entered, the system displays appropriate error messages. The system must allow patient registration, validate login credentials, enforce input validation, and handle authentication errors properly.

3.3.2 Patient Payment (Credits Purchase)

The system shall enable patients to purchase credit packages for booking appointments. Patients can select a desired credit package and proceed to a secure payment gateway. After successful payment, the system updates the patient's credit balance automatically. If the payment fails or is cancelled, the system displays an error message and maintains the existing balance. The system must allow selection of credit packages, integrate a secure payment mechanism, update credits after successful payment, and handle failed or cancelled transactions gracefully.

3.3.3 Patient Find Doctor

The system shall allow patients to search for doctors using criteria such as specialization, name, or availability. When a patient enters search criteria, the system displays a list of matching doctors along with their profiles, ratings, and available time slots. Patients can select a doctor to view detailed information. If no matching doctor is found, the system must notify the patient. The system must support search functionality, display doctor profiles, and handle empty search results.

3.3.4 Patient Book Appointment

The system shall allow patients to schedule appointments with available doctors. Patients select a doctor and choose a time slot from available options. The system verifies availability before confirming the appointment. After confirmation, the system updates schedules for both patient and doctor and sends notifications to both parties. If a time slot is unavailable, the system provides alternative available options. The system must validate availability, confirm bookings, update schedules, and notify users.

3.3.5 Patient Video Call with Doctor

The system shall provide real-time video consultation between patients and doctors. Patients can initiate a video call at the scheduled appointment time, and the system connects a secure session. The video interface includes controls for audio mute, video toggle, camera switching, and ending the call. The system logs call start and end times. In case of connectivity issues, the system displays an error message and provides retry options. The system must support secure video calls, call controls, and session logging.

3.3.6 Doctor Login and Signup

The system shall allow doctors to register and log in to access their dashboards. During registration, doctors must provide credentials, specialization, and license information. The system validates the provided data before account creation. After successful login, doctors are redirected to the doctor dashboard. If login fails, appropriate error messages are displayed. The system must support doctor registration, credential validation, and secure authentication.

3.3.7 Doctor Dashboard

The system shall provide doctors with a dashboard displaying appointments, patient details, and earnings information. After logging in, doctors can view upcoming appointments, consultation status, and credit or payout information. The dashboard must update dynamically as appointments are booked or completed. The system must display appointment details, patient information, and earnings overview.

3.3.8 Doctor Request Payout

The system shall allow doctors to request payouts for completed consultations. Doctors can submit payout requests through the dashboard. The system validates the available balance before processing the request. After submission, the system records the request and confirms the transaction. The system must support payout requests, validate available balance, and maintain payout records.

3.3.9 Doctor Set Availability

The system shall allow doctors to manage their available time slots for appointments. Doctors can add, update, or remove availability slots through their dashboard. The system updates the availability in real-time so that patients can book only available slots. The system must support availability management and update scheduling data accordingly.

3.3.10 Admin Verify and Manage Doctors

The system shall allow administrators to verify doctor credentials and manage doctor profiles. Admin users can view pending doctor registrations, review license information, and approve or reject applications. The admin can also edit or manage doctor profiles when necessary. The system must support doctor verification and profile management.

3.3.11 Admin Payout Management

The system shall allow administrators to manage doctor payouts. Admin users can review payout requests, process payments, and update payout records. The system maintains payout history for transparency and tracking. The system must process payouts, update records, and store payout history.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the MediBot Platform must meet to ensure a reliable, secure, and efficient user experience. These requirements describe how the system performs its functions rather than what it does. They focus on aspects like security, usability, performance, maintainability, and scalability that contribute to the system's overall quality and long-term sustainability.

3.4.1 Performance Requirements

The MediBot platform must provide fast and efficient response times for critical operations such as doctor search, appointment booking, dashboard loading, and video consultation initialization. Under normal load conditions, the system should respond within 2–3 seconds for user actions. Backend APIs and database queries must be optimized to handle concurrent

requests efficiently. Caching techniques should be applied for frequently accessed data such as doctor listings, availability schedules, and user profiles to improve performance and reduce server load.

3.4.2 Scalability Requirements

The MediBot system architecture must be scalable to accommodate future growth in users, doctors, appointments, and consultations. The backend infrastructure built with Next.js server components, Prisma ORM, and PostgreSQL should support horizontal and vertical scaling. The frontend developed using React 19 and Tailwind CSS must efficiently handle large datasets such as doctor listings and appointment history without performance degradation. Cloud-based deployment should allow seamless scaling of compute resources when user demand increases.

3.4.3 Security Requirements

Security is a critical component of the MediBot platform as it handles sensitive patient and doctor information. The system must use secure authentication mechanisms through Clerk Authentication with role-based authorization. All communication between client and server must occur over HTTPS to prevent data interception. User credentials must be securely stored using encrypted authentication services. Input validation and sanitization must be implemented to prevent common vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). Additionally, secure video communication channels must be used for consultations.

3.4.4 Usability Requirements

The MediBot interface must be intuitive, responsive, and easy to use for patients, doctors, and administrators. The design should follow accessibility guidelines and ensure clear navigation across dashboards. Buttons, search filters, booking options, and call controls must be clearly labeled and logically arranged. The Patient, Doctor, and Admin panels should provide consistent layouts and immediate feedback for user actions such as booking appointments, updating availability, or requesting payouts. The system should be responsive across multiple screen sizes.

3.4.5 Reliability Requirements

The MediBot system must remain stable and operational even during high traffic or temporary network disruptions. The platform should aim for at least 99% uptime to ensure

continuous access to appointment booking, dashboards, and video consultations. Error-handling mechanisms must be implemented to manage failed API calls or network interruptions. Automatic retries and fallback mechanisms should ensure minimal disruption to ongoing sessions.

3.4.6 Maintainability Requirements

The system must be modular and easy to maintain. Each module, including authentication, appointment management, credit handling, video consultation, and admin management, should be developed independently. Proper documentation, consistent naming conventions, and clean code structure must be followed. Version control should be maintained using Git to support collaboration and future updates. This modular approach ensures that new features or bug fixes can be implemented without affecting other components.

3.4.7 Portability Requirements

The MediBot application must be platform-independent and accessible through modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. The system must function efficiently across devices including desktops, laptops, tablets, and smartphones. Responsive design using Tailwind CSS should ensure consistent layout and usability across all screen sizes.

3.4.8 Data Integrity Requirements

Data stored in the MediBot system, including patient profiles, doctor information, appointments, and credit balances, must remain accurate and consistent. Validation must be applied at both client and server levels to prevent duplicate entries or incomplete records. Database constraints and transaction handling should ensure consistency during appointment booking and credit deductions. Backup mechanisms must be implemented to restore data in case of system failure.

3.4.9 Availability Requirements

The MediBot system must be deployed on reliable cloud infrastructure to ensure 24/7 availability. The platform should remain accessible for appointment booking, doctor availability management, and video consultations at all times. Load balancing and distributed deployment strategies should be used to minimize downtime. Scheduled maintenance should be communicated in advance to users.

3.4.10 Extensibility Requirements

The system architecture must support future enhancements without major modifications. The modular design should allow easy integration of additional features such as AI-based symptom analysis, chat-based consultation, prescription management, or analytics dashboards. Third-party API integrations should be flexible so that new services can be added without disrupting existing functionality.

3.4.11 Backup and Recovery Requirements

The MediBot system must perform periodic backups of critical data, including user accounts, doctor profiles, appointments, consultation records, and credit balances. Backup data must be stored securely and be easily retrievable. In case of system failure, the recovery mechanism should restore services with minimal data loss and minimal downtime.

3.4.12 Compliance and Legal Requirements

The platform must follow data privacy and security standards for handling medical-related information. User data must be collected only for legitimate purposes and must not be shared without authorization. The system should ensure secure storage of personal data and protect communication between patients and doctors. Access control policies must ensure that only authorized users can view sensitive information.

3.4.14 Load Handling Requirements

The MediBot system should handle concurrent access from at least 500 users simultaneously without noticeable delay or performance degradation. The platform should efficiently manage multiple appointment bookings and concurrent video sessions. Performance testing tools such as Postman or Apache JMeter should be used to validate system performance before deployment.

Chapter 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

In the Design and Architecture phase of our MediBot Healthcare System, we establish the overall structural framework of the application and define how its components interact to deliver a seamless healthcare experience. This phase focuses on designing intuitive user interfaces for patients, doctors, and administrators, while also developing a robust backend architecture that efficiently manages user data, appointment scheduling, credit-based payments, and real-time video consultations. The system is designed to ensure smooth communication between all modules, secure data handling, and scalable performance for multiple users.

4.1 System Architecture

Modular Program Structure:

MediBot is organized into a set of well-defined, interconnected modules, where each module is responsible for a distinct aspect of the system's functionality. This modular approach allows the system to efficiently manage the complex workflows of a telemedicine platform, including user authentication, doctor verification, appointment scheduling, credit-based transactions, video consultations, and administrative oversight. By separating responsibilities into discrete modules, MediBot ensures that each component can be developed, maintained, and updated independently without affecting other parts of the system.

The modular structure also facilitates clear communication and data flow between patients, doctors, and administrators. For instance, when a patient books an appointment, the request passes seamlessly from the patient module to the appointment scheduling module, triggers credit deduction in the payment module, notifies the doctor through the notification module, and updates the doctor's dashboard. Similarly, doctor verification requests flow through the admin module for approval, which then enables access to the doctor panel.

1. **User Authentication Module:** Manages secure login, signup, and role-based access using Clerk.
2. **Patient Module:** Allows patients to buy credits, search doctors, book appointments, and join video calls.
3. **Doctor Module:** Enables doctors to verify profiles, manage availability, view appointments, and conduct video consultations.
4. **Admin Module:** Handles doctor verification, payouts, system monitoring, and account management.
5. **Appointment & Scheduling Module:** Manages booking, slot selection, rescheduling, and appointment tracking.
6. **Credits & Payment Module:** Processes payments, assigns credits, deducts credits for bookings, and credits doctor accounts.
7. **Video Consultation Module:** Provides secure patient-doctor video calls using Vonage API.
8. **Database Module:** Stores system data such as users, appointments, payments, and availability using Neon + Prisma.

Relationships Between Modules:

- Patients interact with the Authentication, Patient, Appointment, Payment, and Video Consultation modules.
- Doctors interact with the Authentication, Doctor, Appointment, Video Consultation, and Credits modules.
- Admin interacts with the Admin, Verification, Payout, and Analytics-related backend logic.
- The Appointment & Scheduling module acts as the connecting bridge between patients and doctors.

- The Credits & Payment module connects patient purchases with doctor payouts.
- The Database module (Neon + Prisma) serves as a shared data layer allowing all modules to read/write securely

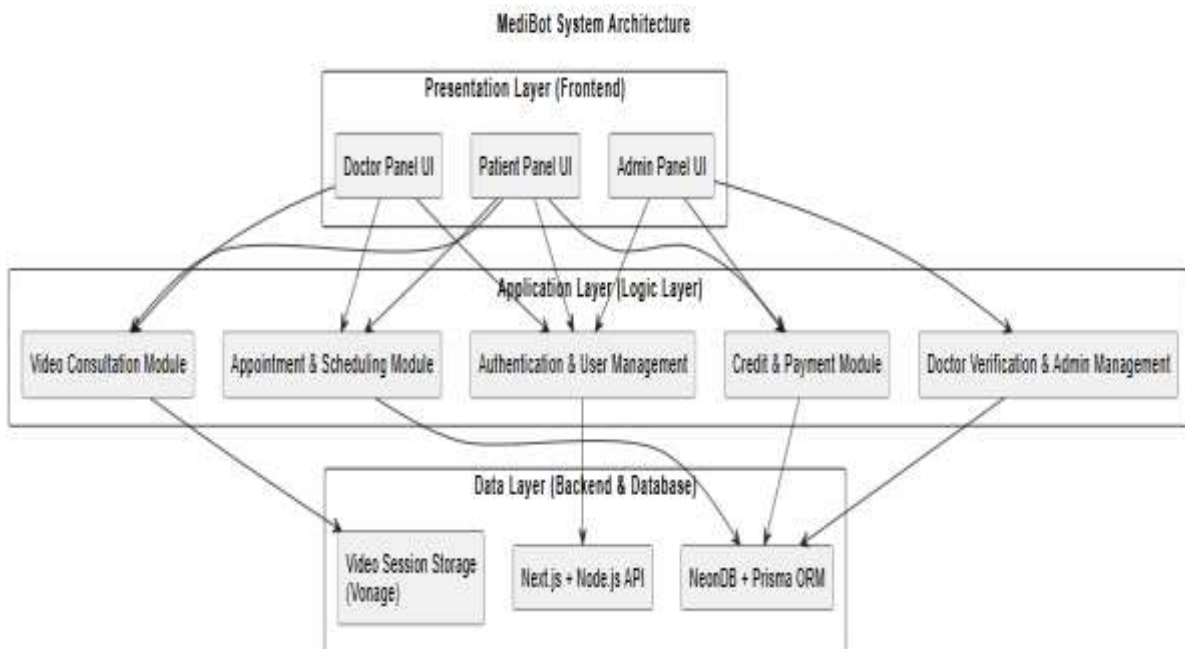


Figure 4. 1: MediBot System architecture

The diagram illustrates the system architecture of the MediBot application, which is organized into three main layers: Presentation Layer, Application Layer, and Data Layer. The Presentation Layer (Frontend) includes three main user interfaces: Doctor Panel UI, Patient Panel UI, and Admin Panel UI, allowing different users to interact with the system. These interfaces communicate with the Application Layer (Logic Layer), where the core functionalities of the system are handled. This layer contains modules such as Video Consultation, Appointment & Scheduling, Authentication & User Management, Credit & Payment, and Doctor Verification & Admin Management. Each module processes user requests and manages the system's operations. The Data Layer (Backend & Database) contains components like Video Session Storage (Vonage), Next.js + Node.js API, and NeonDB + Prisma ORM, which handle data storage and retrieval.

stores and manages all application data and services. It includes technologies like Next.js with Node.js API, NeonDB with Prisma ORM for database management, and Vonage for video session storage. Together, these layers ensure smooth communication, secure data handling, and efficient functioning of the MediBot healthcare platform.

4.2.2 ER Diagram

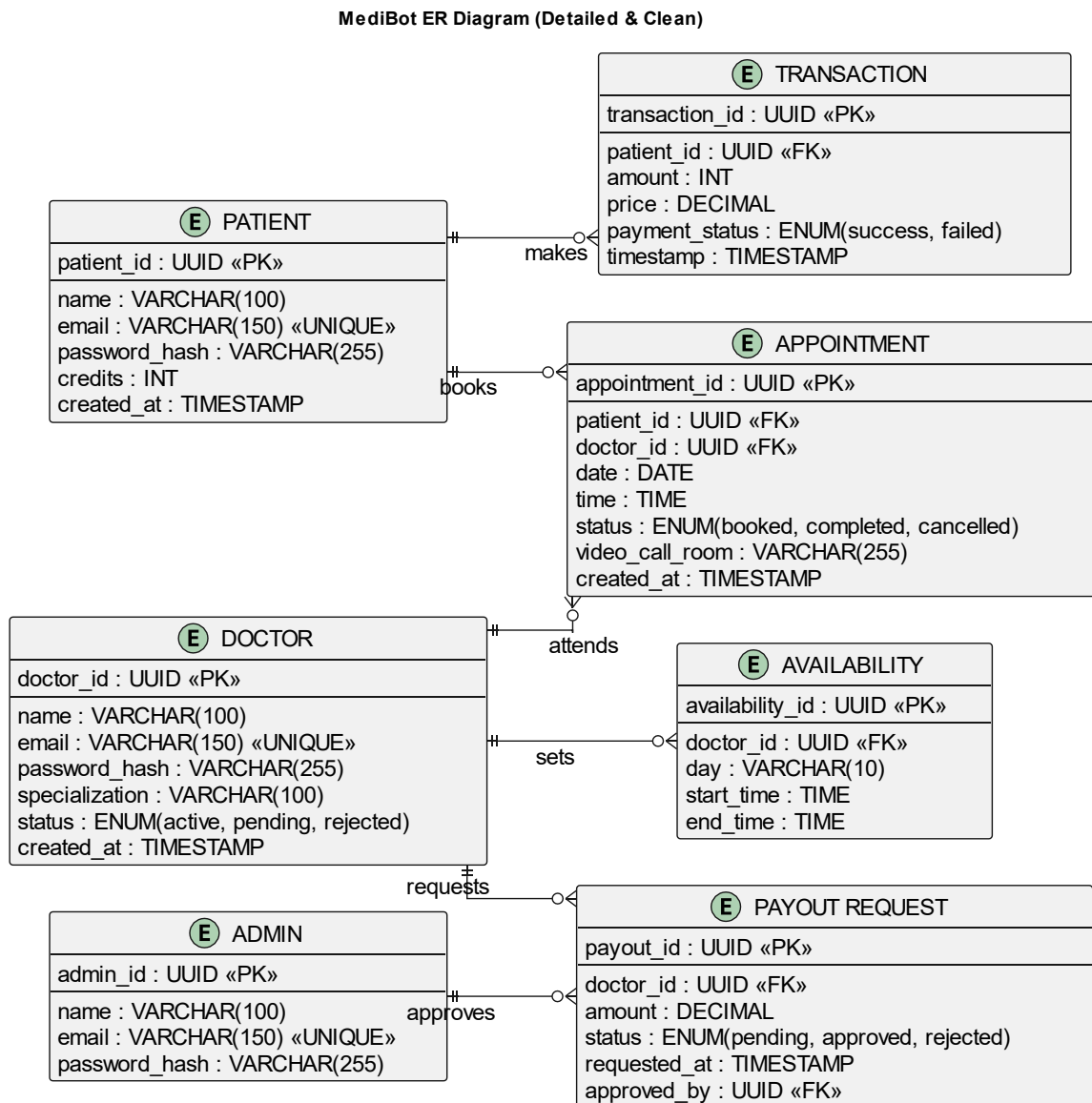


Figure 4. 2: MediBot ER Diagram

The ER illustrates the core data structure of the MediBot system, showing the main entities such as Patient, Doctor, Admin, Appointment, and Payout, along with their relationships. It highlights how patients book appointments with doctors, how credits are managed for payments, and how doctors request payouts. The diagram also represents key associations like one-to-many relationships between users and appointments, and doctors with availability schedules. Overall, the ER provides a clear view of how data is organized, stored, and interconnected within the system.

4.3 Process Flow/Representation

The **MediBot platform** enables users to access healthcare services efficiently through a structured and user-friendly workflow. Patients can register or log in, purchase credits, search for doctors based on specialization, view doctor profiles, and book appointments by selecting available time slots. Once an appointment is confirmed, patients can connect with doctors through real-time video consultations.

Doctors can log in to the system, manage their availability, view scheduled appointments, and conduct video consultations with patients. They can also request payouts for the credits earned through completed consultations. Administrators play a key role in managing the system by verifying doctor registrations, monitoring platform activities, handling payout approvals, and ensuring system integrity.

Overall, the system integrates patient services, doctor management, and administrative control into a unified platform, ensuring efficient healthcare delivery, smooth communication, and an improved user experience.

4.3.1 Activity Diagrams

These diagrams effectively illustrate the sequence of steps involved in each workflow of the MediBot system, highlight key decision points such as appointment booking confirmation, credit availability, and doctor verification, and reveal parallel processes such as updating schedules while managing ongoing consultations. By visualizing the system's workflows, activity diagrams improve communication between developers and

stakeholders, support accurate system design and implementation, and ensure that the platform's logic and interactions are clearly understood throughout development. They also help reduce ambiguity by identifying potential issues early in the design phase. Additionally, these diagrams provide a strong foundation for building a system that is efficient, reliable, and user-friendly, while meeting the needs of patients, doctors, and administrators.

Major Process Flow:

1. Patient buying credits & booking appointment.
2. Doctor managing availability and conducting video consultation.
3. Admin verifying doctors and approving payouts.

4.3.1.1 Patient Buying Credits & Booking Appointment

This activity diagram represents the workflow followed by a patient from purchasing credits to successfully booking an appointment. The process begins with the patient logging into the system and selecting a suitable credit package. After completing the payment, credits are added to the patient's account. The patient then searches for a doctor based on specialization, views available time slots, and selects a preferred slot. Before confirming the booking, the system checks whether sufficient credits are available and whether the selected slot is still open. If all conditions are satisfied, the appointment is successfully booked, and the patient receives confirmation along with details for the upcoming video consultation.

4.3.1.2 Doctor Managing Availability and Conducting Video Consultation

This activity diagram illustrates the workflow for doctors in managing their schedules and interacting with patients. The process starts when the doctor logs into the system and sets or updates availability slots. Patients can book appointments based on these slots. The doctor can view upcoming appointments through the dashboard and prepare for scheduled consultations. At the designated time, the doctor joins the video session and conducts the consultation with the patient. After the session is completed, the appointment status is updated, and the doctor earns credits for the consultation, which can later be requested for payout.

4.3.1.2 Admin Verifying Doctors and Approving Payouts

This activity diagram shows the administrative workflow for maintaining system integrity and managing financial operations. The process begins with the admin reviewing doctor registration requests, including submitted documents for verification. Based on the evaluation, the admin either approves or rejects the doctor's account. Additionally, the admin monitors payout requests submitted by doctors. Each request is reviewed for validity, and upon approval, the payment is processed. If any discrepancies are found, the request may be rejected. This workflow ensures that only verified doctors operate on the platform and that all financial transactions are properly controlled and authorized.

4.3.1.3 Patient: Buying Credits & Booking Appointment Activity Diagram



Figure 4. 3: Patient: Buying Credits & Booking Appointment Activity Diagram

Figure 4.3 this activity diagram illustrates the complete workflow of a patient purchasing credits and booking an appointment in the MediBot system. The process begins with the patient checking their current credit balance, followed by selecting a suitable credit package and completing the payment through the integrated payment system. Once the credits are successfully added to the account, the patient searches for doctors based on specialization and reviews their profiles. After selecting a preferred doctor, the patient chooses an available appointment slot. The system then deducts the required credits and confirms the booking. Finally, the patient receives an appointment confirmation, marking the successful completion of the process.

4.3.1.4 Doctor: Managing Availability & Video Consultation Activity Diagram

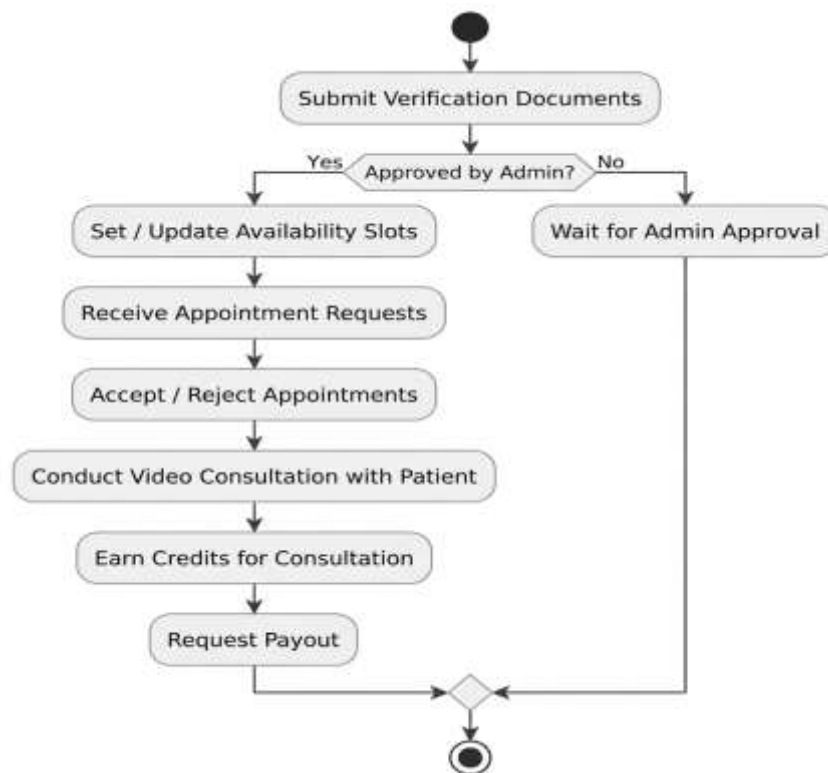


Figure 4. 4: Doctor: Managing Availability & Video Activity Diagram

Figure 4.4 this activity diagram illustrates the workflow of a doctor within the MediBot system, starting from the verification process to earning and requesting payouts. The process begins

when the doctor submits verification documents for approval. If the admin approves the request, the doctor can proceed to set or update availability slots. The doctor then receives appointment requests from patients and can choose to accept or reject them. For accepted appointments, the doctor conducts video consultations and earns credits upon completion. These earned credits can later be converted into payouts by submitting a payout request. If the doctor is not approved initially, they must wait for admin approval before accessing system functionalities. This workflow ensures that only verified doctors can provide services and participate in the platform.

4.3.1.5 Admin: Verifying Doctors & Approving Payouts Activity Diagram

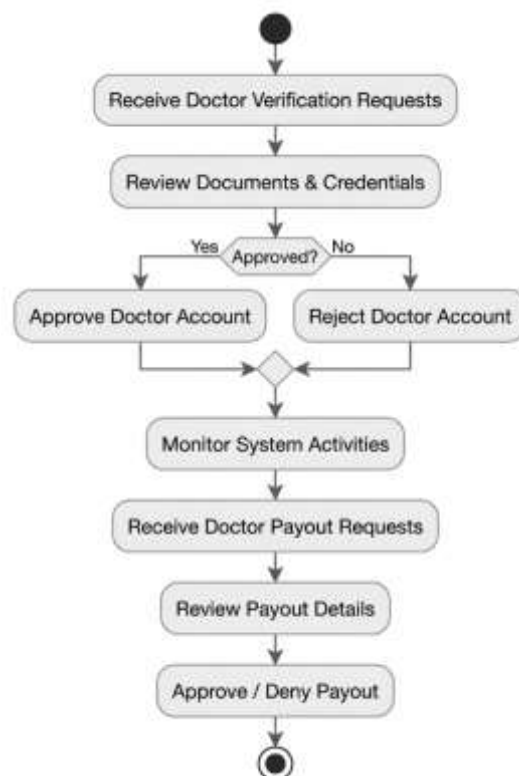


Figure 4. 5: Admin: Verifying Doctors & Approving Payouts Activity Diagram

Figure 4.5 this activity diagram represents the administrative workflow in the MediBot system for managing doctor verification and payout approvals. The process begins when the admin

receives doctor verification requests and reviews the submitted documents and credentials. Based on the evaluation, the admin either approves or rejects the doctor's account. After this decision, the admin continues to monitor overall system activities to ensure smooth platform operation. The admin also handles payout requests submitted by doctors by reviewing the payout details and either approving or denying the request.

4.4 Design Models

4.4.1 Sequence Diagram

Sequence Diagrams are interaction diagrams that illustrate how different components of the MediBot system communicate with each other to perform specific operations. They capture the interaction between system entities such as Patient, Doctor, Admin, and backend modules within a particular workflow. Sequence diagrams are time-oriented, showing the order of interactions through a vertical timeline, where messages are exchanged between components.

4.4.1.1 Sequence Diagram for Patient Core Process

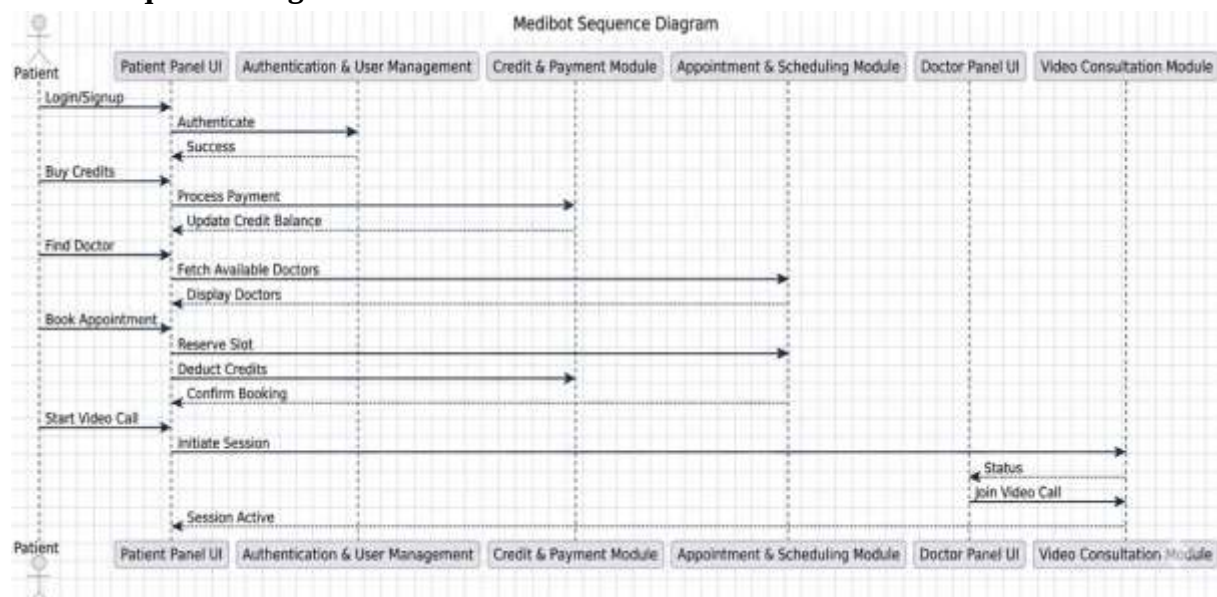


Figure 4. 6: Sequence Diagram for Patient Core Process

Figure 4.6 this diagram represents the complete patient workflow in MediBot. It covers login/signup, purchasing credits, searching for doctors, booking appointments, and

participating in video consultations. The diagram highlights how the Patient Panel UI interacts with backend modules like Authentication, Credit & Payment, and Appointment & Scheduling, as well as with the Doctor Panel via video sessions, illustrating the seamless flow of actions and data across the system.

4.4.1.2 Sequence Diagram for Doctor Core Process

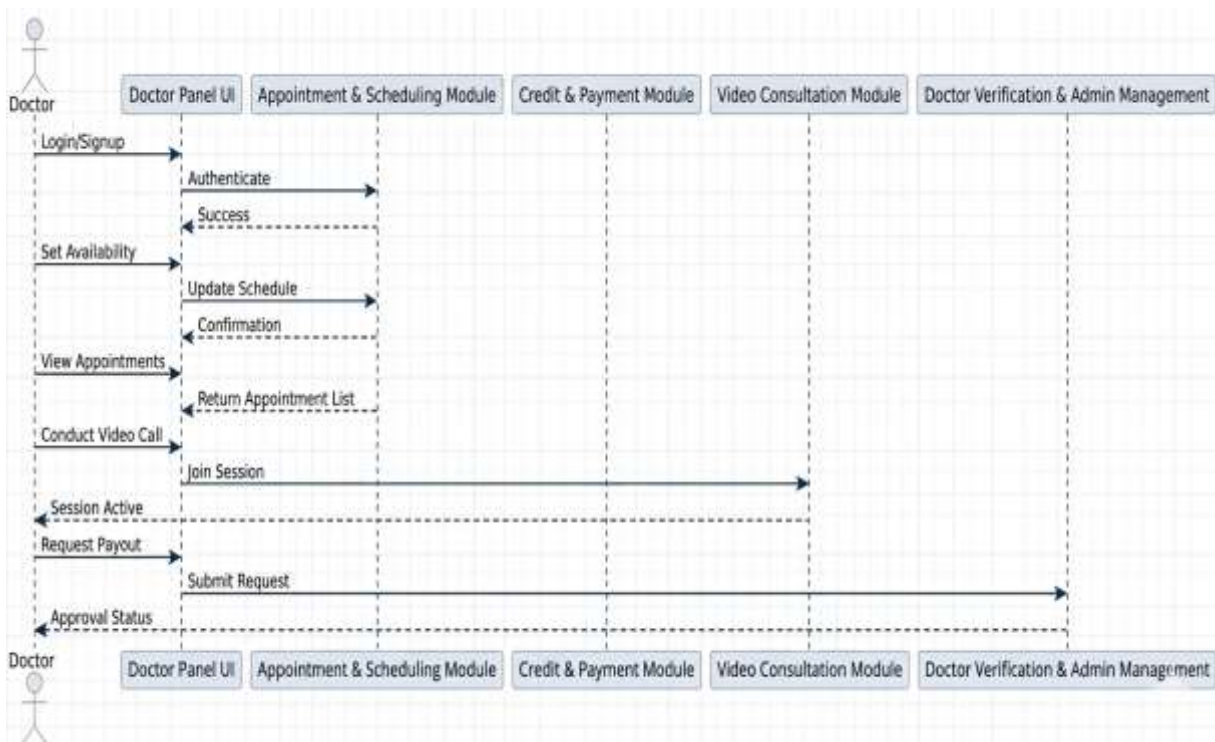


Figure 4. 7: Sequence Diagram for Doctor Core Process

Figure 4.7 this diagram illustrates the doctor's core workflow in MediBot. It includes login/signup, setting availability, viewing scheduled appointments, conducting video consultations, and requesting payouts. The diagram highlights how the Doctor Panel UI interacts with backend modules like Appointment & Scheduling, Credit & Payment, Video Consultation, and Admin Management, ensuring that all actions are properly synchronized and data remains consistent across the system.

4.4.1.3 Sequence Diagram for Admin Core Process

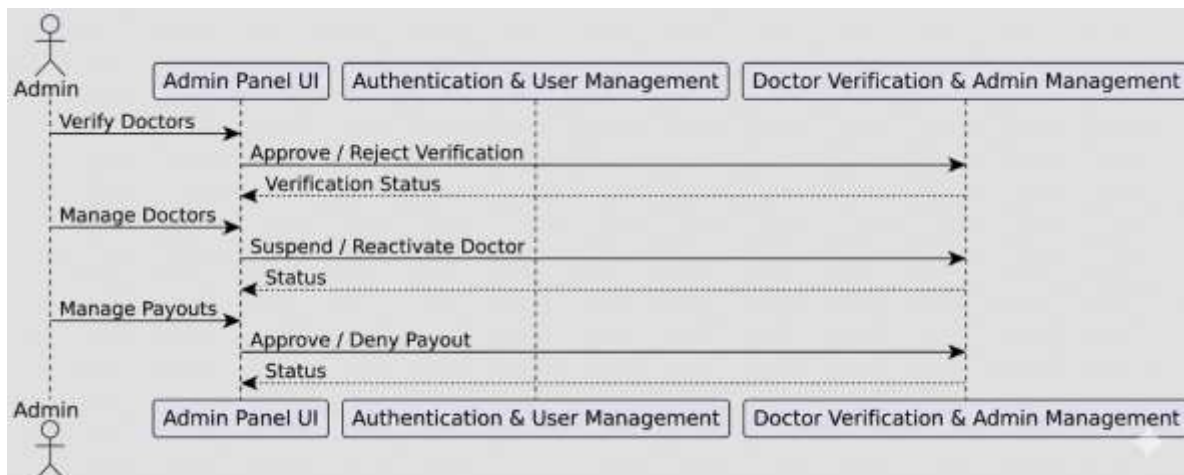


Figure 4. 8: Sequence Diagram for Admin Core Process

Figure 4.8 this diagram illustrates the admin's core workflow in MediBot, detailing the critical oversight tasks required to maintain the platform's integrity and operational standards. The process begins with the admin interacting through the Admin Panel UI to manage high-level governance functions, such as the verification of medical professionals. By interfacing with the Doctor Verification & Admin Management module, the admin can approve or reject verification requests, ensuring that only qualified doctors are permitted to practice.

4.4.2 Class Diagram

A class diagram is a static structure diagram in UML that represents the overall structure of the MediBot system by illustrating its core classes, attributes, methods, and the relationships between them. It is widely used in object-oriented design to visualize system architecture, support documentation, and guide implementation. In MediBot, the class diagram includes key entities such as User, Patient, Doctor, Admin, Appointment, Credit Transaction, Availability, and Payout Request.

The diagram provides a clear representation of inheritance relationships, where Patient, Doctor, and Admin are derived from the base User class, as well as associations such as patients booking appointments with doctors and doctors managing availability schedules. It also highlights how credits and payments are linked to appointments and how admins control verification and payout processes.

Overall, the class diagram presents the static view of the MediBot application, helping developers understand system structure, ensure proper organization of data and logic, and support efficient development of healthcare functionalities within the platform.

4.4.2.1 Class Diagram

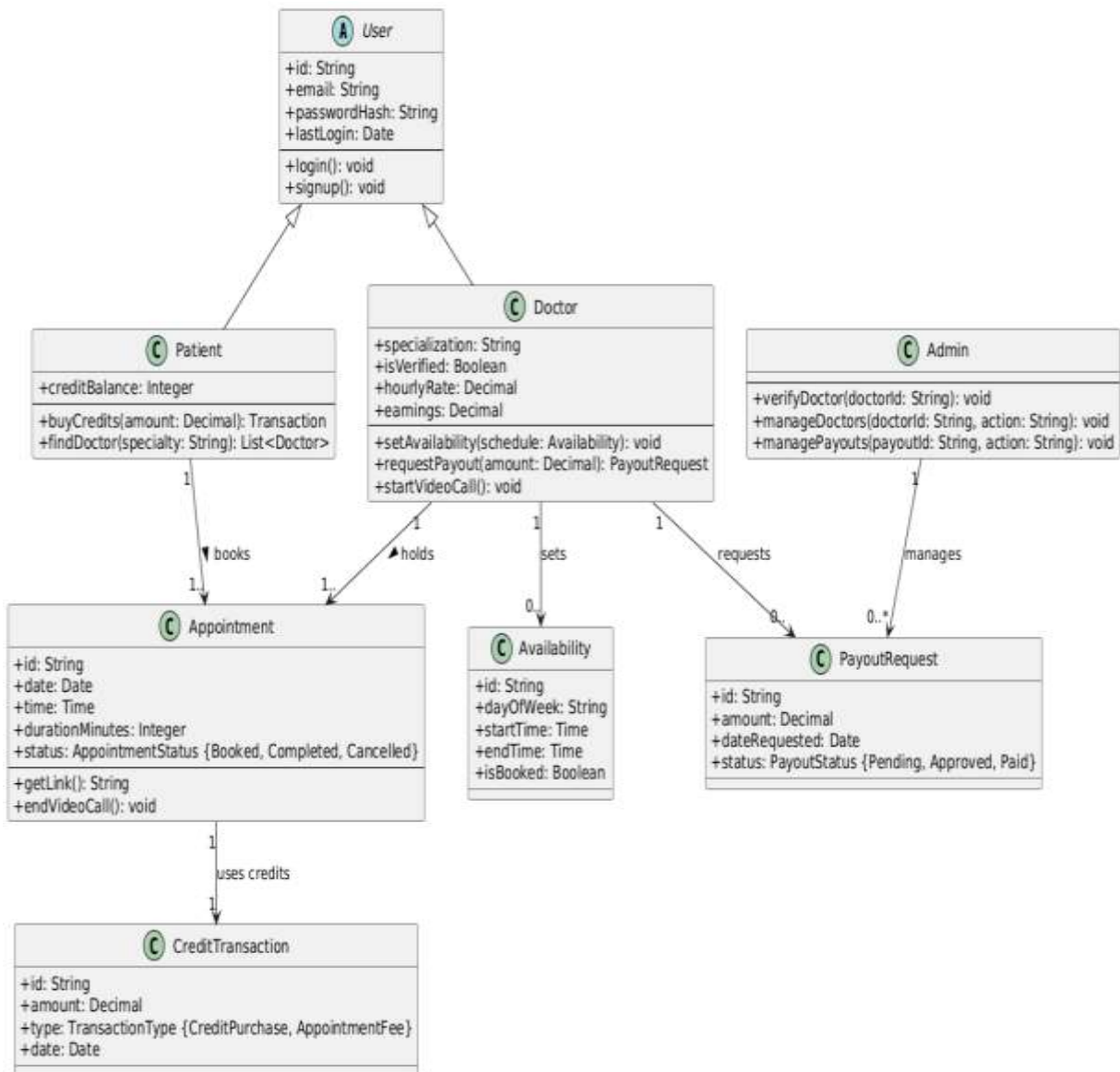


Figure 4. 9: Class Diagram

Figure 4.9: Class Diagram of MediBot System

The class diagram of the MediBot system represents the core entities, their attributes, methods, and the relationships between them. It provides a structural overview of how different components of the MediBot platform interact to deliver AI-powered healthcare services such as symptom analysis, appointment scheduling, and medical guidance.

Chapter 5

IMPLEMENTATION

5 Implementation

In the implementation phase of MediBot, we have developed a set of well-defined algorithms to power the core functionalities of the application. These algorithms enable accurate appointment scheduling, real-time doctor-patient communication, secure payment processing, and smooth interactions between users, the Node.js backend, and the MongoDB database. Built using React on the frontend and Node.js with Express on the backend, MediBot follows a RESTful API architecture that ensures efficient healthcare operations and a seamless experience for Patients, Doctors, and Administrators.

5.1 Algorithm

The following subsections present the core algorithms implemented in MediBot. Each algorithm is described with its function signature, inputs, outputs, a plain-language description, and a detailed pseudocode walkthrough.

5.1.1 User Authentication (Login / Signup)

Function: `authenticateUser(credentials)`

Input: `credentials` (object) — `email` (string), `password` (string), `role` (string: 'patient' | 'doctor' | 'admin')

Output: `authToken` (string) and `userProfile` (object) or `error` (string)

Description:

This algorithm handles secure registration and login for two actor types in MediBot: Patient, Doctor. On signup, user data is validated and the password is hashed using `bcrypt` before being stored in NeonDB. On login, the entered password is compared against the stored hash. Upon successful authentication, a JSON Web Token (JWT) is generated and returned to the client. React stores this token in memory and attaches it to every subsequent API request via the `Authorization` header, ensuring that only verified users can access protected routes.

Pseudocode:

- Receive email, password, and role from the client request.
- Validate that all fields are non-empty and that the email format is correct.
- Query the Users collection in MongoDB for a document matching the provided email and role.
- If no matching document is found, return error: "User not found."
- If a document is found, use bcrypt.compare() to verify the entered password against the stored hashed password.
- If passwords do not match, return error: "Incorrect password."
- If passwords match, generate a JWT token signed with the server's secret key and an expiry of 7 days.
- Return the JWT token along with the user profile object (name, role, profileImage) to the client.
- On the React frontend, store the token and redirect the user to their role-specific dashboard.

5.1.2 Book Appointment

Function: bookAppointment (patientId, doctorId, slotDateTime)

Input: patientId (string), doctorId (string), slotDateTime (ISO 8601 string)

Output: appointmentConfirmation (object) or error (string)

Description:

This algorithm allows a Patient to book a consultation appointment with a selected Doctor. Before creating the booking, the system checks whether the chosen time slot falls within the Doctor's declared availability and has not already been reserved. If the slot is free, a new

Appointment document is persisted in NeonDB, the slot is marked as taken, and a confirmation notification is dispatched to both the Patient and the Doctor via the notification service.

Pseudocode:

- Receive patientId, doctorId, and slotDateTime from the frontend booking form.
- Fetch the Doctor's availability schedule from the Availability collection using doctorId.
- Check whether slotDateTime falls within any of the Doctor's declared available time windows.
- If slotDateTime is outside available windows, return error: "Doctor not available at this time."
- Query the Appointments collection to verify that no existing appointment already occupies the same doctorId and slotDateTime.
- If a conflict is found, return error: "Slot already booked. Please choose another time."
- Create a new Appointment document with status set to 'pending', and save it to MongoDB.
- Update the Doctor's availability record to mark that slot as booked.
- Dispatch confirmation notifications to both the Patient and the Doctor.
- Return the appointment confirmation object including appointmentId, doctor name, and scheduled time.

5.1.3 Buy Credits / Payment Processing

Function: processPayment(patientId, creditPackage)

Input: patientId (string), creditPackage (object) — packageId, amount, credits

Output: paymentReceipt (object) or error (string)

Description:

This algorithm handles the purchase of consultation credits by a Patient. Credits are the in-app currency used to pay for video call appointments. The algorithm initiates a payment request to the integrated third-party payment gateway, verifies the transaction result, and upon success, updates the Patient's credit balance in NeonDB.

Pseudocode:

- Receive patientId and selected creditPackage details from the React payment form.
- Validate that the package amount is a positive number and that the patientId exists in the database.
- Send a payment initiation request to the payment gateway API with the transaction amount and currency.
- Await the payment gateway response.
- If the response status is 'failed' or 'declined', return error: "Payment unsuccessful. Please try again."
- If the response status is 'success', retrieve the Patient's current credit balance from the Patients collection.
- Add the purchased credits to the existing balance and save the updated document to NeonDB.
- Create a new Payment record in the Payments collection including transactionId, amount, credits, and timestamp.
- Return the payment receipt object and the updated credit balance to the frontend.

5.1.5 Video Call with Doctor

Function: initiateVideoCall(appointmentId, initiatorId)

Input: appointmentId (string), initiatorId (string)

Output: callSession (object) — roomToken, roomId or error (string)

Description:

This algorithm initiates a real-time video consultation between a Patient and a Doctor for a confirmed appointment. When the scheduled appointment time is reached, the system generates a unique video room and access tokens using the Vonage video API. Credits are deducted from the Patient's balance at the start of the session. Both participants receive their respective room tokens and are connected to the secure video session.

Pseudocode:

- Receive appointmentId and initiatorId from the React frontend call button.
- Fetch the Appointment document from NeonDB using appointmentId.
- Verify that the appointment status is 'confirmed' and that the current time is within the scheduled window.
- If the appointment is not confirmed or the time window has not started, return error: "Call cannot be started at this time."
- Check that the Patient has sufficient credits to cover the session cost.
- If credits are insufficient, return error: "Insufficient credits. Please top up your balance."
- Generate a unique roomId for this session.
- Request access tokens for both the Patient and the Doctor from the video API service using the roomId.
- Deduct the required credits from the Patient's balance in NeonDB and log the transaction.
- Update the Appointment status to 'in-progress'.
- Return the roomId and respective tokens to both the Patient and Doctor clients so they can join the video room.

5.1.6 View Appointments

Function: getAppointments(userId, role)

Input: userId (string), role (string: 'patient' | 'doctor')

Output: appointmentsList (array) or error (string)

Description:

This algorithm retrieves all appointments associated with a given user, whether they are a Patient or a Doctor. It queries the Appointments collection using the userId and role, then populates related Doctor or Patient profile data for display.

Pseudocode:

- Receive userId and role from the authenticated session.
- Build a query filter: if role is 'patient', filter by patientId; if role is 'doctor', filter by doctorId.
- Query the Appointments collection with the filter and sort by scheduledAt in ascending order.
- For each appointment, populate the opposing party's profile data (doctor name and photo for patients; patient name for doctors).
- Categorize appointments into three groups: 'upcoming' (scheduled in the future), 'completed', and 'cancelled'.
- Return the categorized appointments list to the React dashboard for rendering.

5.1.7 Set Availability (Doctor)

Function: setAvailability(doctorId, availabilitySlots)

Input: doctorId (string), availabilitySlots (array of objects — day, startTime, endTime)

Output: success (boolean) or error (string)

Description:

This algorithm allows a doctor to define or update their weekly availability schedule. The Doctor selects working days and time ranges through the React dashboard, and the system validates and saves these slots to the Availability collection. Existing slots that are already booked by patients are preserved and cannot be removed, ensuring no disruption to confirmed appointments.

Pseudocode:

- Receive doctorId and the new availability slots array from the React availability form.
- Validate that each slot has a valid day, a non-null startTime, and an endTime that is after startTime.
- Fetch the Doctor's current Availability document from NeonDB.
- Identify any existing slots that are already linked to confirmed or pending appointments.
- Ensure that none of the already-booked slots are being removed or altered in the new submission.
- If a conflict is found, return error: "Cannot remove a slot that has an existing booking."
- Replace the current availability document with the new set of slots, preserving booked slot statuses.
- Save the updated Availability document to NeonDB.
- Return success confirmation to the Doctor's dashboard.

5.1.8 Request Payout (Doctor)

Function: requestPayout(doctorId, amount)

Input: doctorId (string), amount (number)

Output: payoutRequest (object) or error (string)

Description:

This algorithm allows a Doctor to request a withdrawal of their earned earnings. The system verifies that the requested amount does not exceed the Doctor's available balance — calculated from completed consultation fees — and creates a payout request record with a 'pending' status. The Admin reviews and approves payout requests from the Admin dashboard, after which the funds are disbursed.

Pseudocode:

- Receive doctorId and requested payout amount from the Doctor's earnings dashboard.
- Validate that the amount is a positive number greater than zero.
- Retrieve the Doctor's earnings record from the Earnings collection and calculate the available withdrawable balance.
- If the requested amount exceeds the available balance, return error: "Requested amount exceeds available balance."
- Create a new Payout document with status 'pending', storing doctorId, amount, and requestTimestamp.
- Deduct the requested amount from the Doctor's available balance and update the Earnings record.
- Notify the Admin of the new pending payout request via the notification service.
- Return the payout request object including payoutId and estimated processing time.

5.1.9 Manage Payouts (Admin)

Function: managePayout(adminId, payoutId, action)

Input: adminId (string), payoutId (string), action (string: 'approve' | 'reject')

Output: success (boolean) or error (string)

Description:

This algorithm enables the Admin to review, approve, or reject Doctor payout requests. Upon approval, the system triggers a disbursement call to the payment gateway and updates the payout status accordingly. If a request is rejected, the deducted amount is restored to the Doctor's available balance, and the Doctor is notified of the reason.

Pseudocode:

- Receive adminId, payoutId, and action from the Admin dashboard.
- Verify that the requesting user has Admin privileges using their JWT role claim.
- Fetch the Payout document from MongoDB using payoutId and confirm its status is 'pending'.
- If the document is not found or status is not 'pending', return error: "Invalid or already processed payout request."
- If action is 'approve': send a disbursement request to the payment gateway with the payout amount and doctor's bank details.
- On gateway success, update the Payout document status to 'approved' and record the disbursementId.
- If action is 'reject': restore the deducted amount to the Doctor's available balance in the Earnings collection.
- Update the Payout document status to 'rejected' and store the rejection reason.
- Notify the Doctor of the payout outcome via the notification service.
- Return success confirmation to the Admin dashboard.

5.2 External APIs

Table 5. 1: External APIs

Library / API	Purpose
Axios	Handles REST API requests between frontend and backend services.
Next.js API Routes	Provides backend endpoints for authentication, appointments, credits, and dashboard operations.
PostgreSQL	Stores patient, doctor, appointment, and transaction data.
Prisma ORM	Database ORM used for modeling and interacting with PostgreSQL.
Clerk Authentication	Secure user authentication and role-based access control for patients, doctors, and admin.
Vonage Video API	Enables real-time video consultations between patients and doctors.
Shadcn UI	Provides reusable UI components for dashboards and forms.
Tailwind CSS	Used for responsive and modern user interface styling.

Table 15.2.1: lists the key libraries and APIs utilized in the MediBot project along with their specific roles in the system. Axios is used to manage communication between frontend components and backend APIs. Next.js API routes provide server-side functionality for handling appointments, credit management, authentication, and administrative operations. PostgreSQL serves as the primary database for storing user accounts, doctor profiles, availability schedules, consultation history, and credit transactions, while Prisma ORM simplifies database modeling and query execution.

Clerk Authentication is used for secure login, signup, and role-based access control for patients, doctors, and administrators. The Vonage Video API enables real-time video consultations with secure audio and video communication features. Payment gateway integration is used to support the credit-based booking model, allowing patients to purchase credits securely. Tailwind CSS and Shadcn UI are used to design responsive and modern dashboards, ensuring

a consistent user experience across all modules. Additionally, email services such as Nodemailer are used to send appointment confirmations, doctor verification notifications, and payout alerts. The system is deployed on cloud infrastructure such as Vercel, ensuring scalability, performance, and accessibility.

5.3 User Interface

The MediBot telemedicine platform is designed using a user-centric approach to ensure ease of use for patients, doctors, and administrators. The interface is modern, responsive, and intuitive, allowing users to navigate the system without requiring technical expertise. The layout is structured into role-based dashboards, where each user type interacts with features relevant to their responsibilities. The system follows a clean design with clearly labeled buttons, structured forms, and smooth navigation to ensure a seamless experience.

The Patient Interface allows users to register, log in, search for doctors, purchase credits, book appointments, and join video consultations. The dashboard provides an overview of upcoming appointments, credit balance, and consultation history. Patients can easily filter doctors by specialization, view doctor profiles, and select available time slots. The booking workflow is simplified into step-by-step actions, making it easy for users to complete consultations. The video consultation interface includes controls for microphone, camera, and call termination, ensuring smooth communication.

The Doctor Interface provides a dedicated dashboard where doctors can manage their availability, view scheduled appointments, and monitor earnings. Doctors can add or modify available time slots, accept consultation requests, and conduct video consultations directly within the platform. The dashboard also displays patient details, consultation status, and payout information. The interface is designed to reduce complexity and allow doctors to focus on managing consultations efficiently.

The Admin Interface provides centralized control over the entire platform. Administrators can verify doctor registrations, manage users, monitor consultations, and handle payout requests. The admin dashboard includes analytics panels, user management tables, and verification workflows. Admin users can approve or reject doctor applications and manage system data efficiently. The interface ensures that administrative tasks can be completed quickly and accurately.

The entire MediBot user interface is fully responsive and compatible across desktops, laptops, tablets, and mobile devices. Navigation menus, forms, and dashboards automatically adjust to different screen sizes. The consistent design across patient, doctor, and admin panels ensures a smooth user experience while maintaining clarity and usability throughout the system

5.3.1 Sign-in Screen

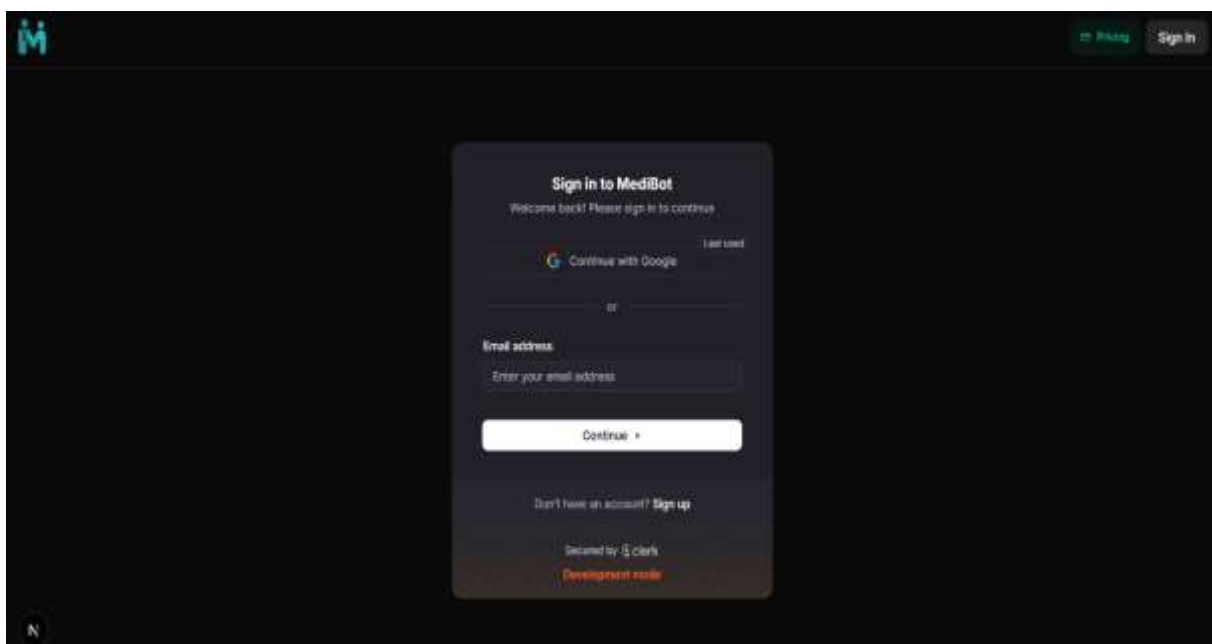


Figure 5. 1: Sign-in Screen

Figure 5.1: The image shows the Sign in to MediBot page where existing users can log into the system. Users are welcomed back and prompted to sign in to continue using the

platform. They have the option to quickly log in using their Google account by selecting the “Continue with Google” button. Alternatively, users can manually enter their email address in the provided field. After entering the email, they can click the “Continue” button to proceed with the authentication process. If a user does not already have an account, they can select the “Sign up” link at the bottom of the page to create a new account. The email and password are authenticated by clerk and once both fields are true the user logged in

5.3.2 Sign-up Screen

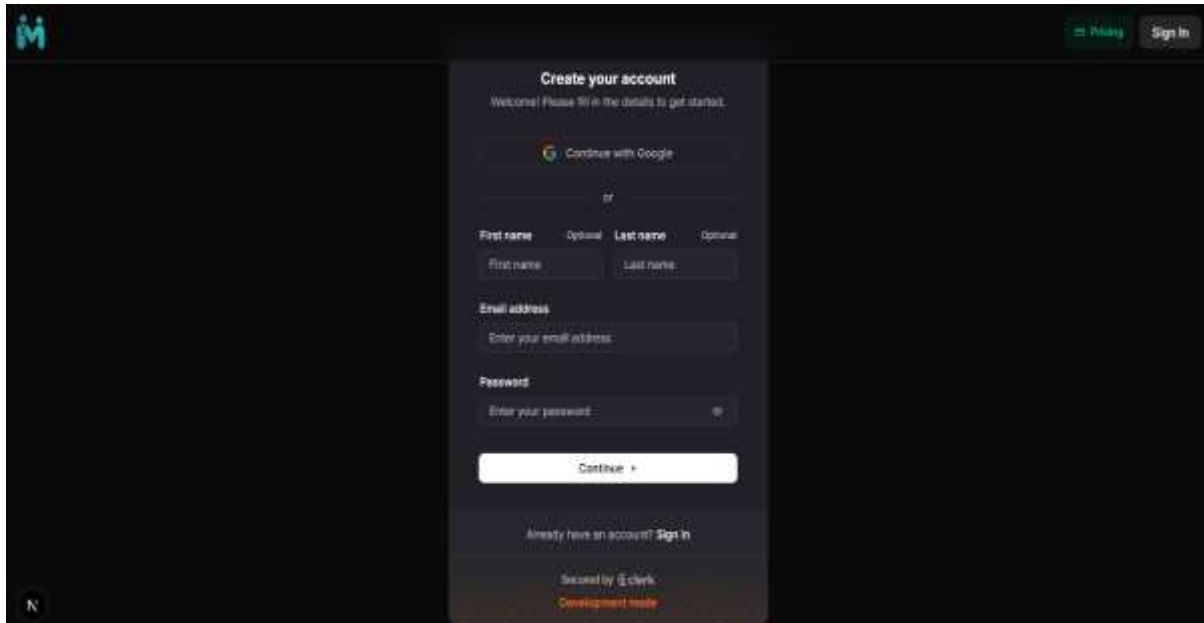


Figure 5. 2: Sign-up Screen

Figure 5.2: The image shows the Home Screen of the MediBot platform, which serves as the main interface after the user logs into the system. The page displays the heading “Connect with doctors anytime, anywhere,” highlighting the platform’s purpose of providing easy access to healthcare services. It informs users that they can book appointments, consult doctors through video calls, and manage their healthcare journey in one secure platform. Two main buttons, “Get Started” and “Find Doctors,” allow users to quickly begin using the services or search for available doctors. At the top navigation bar, users can also view options such as My

Appointments and their available credits, providing quick access to their scheduled consultations and account information.

5.3.3 Doctor Profile and Appointment Booking Screen

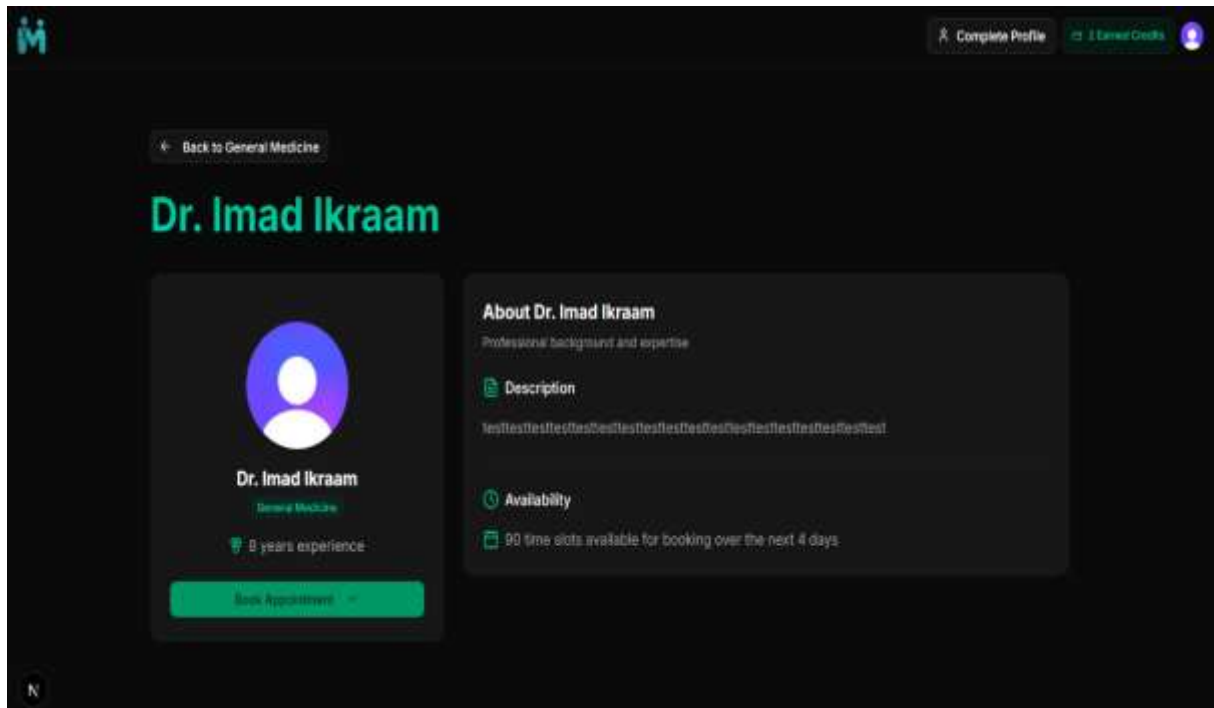


Figure 5. 3: Doctor Profile and Appointment Booking Screen

Figure 5.3: This screen represents a Doctor Profile and Appointment Booking interface within a healthcare platform. It displays essential information about the doctor, including their name, specialization, professional experience, and a brief description of their background. The profile section helps patients quickly understand the doctor's expertise and medical field. Along with the doctor's details, the screen also provides availability information, showing the number of open time slots for upcoming days. A “Book Appointment” button is prominently placed to allow users to easily schedule a consultation. The layout is designed with a clean, dark-themed interface to improve readability and user experience. Overall, this screen enables patients to review a doctor's profile and conveniently book appointments online.

5.3.4 Doctor Dashboard Screen

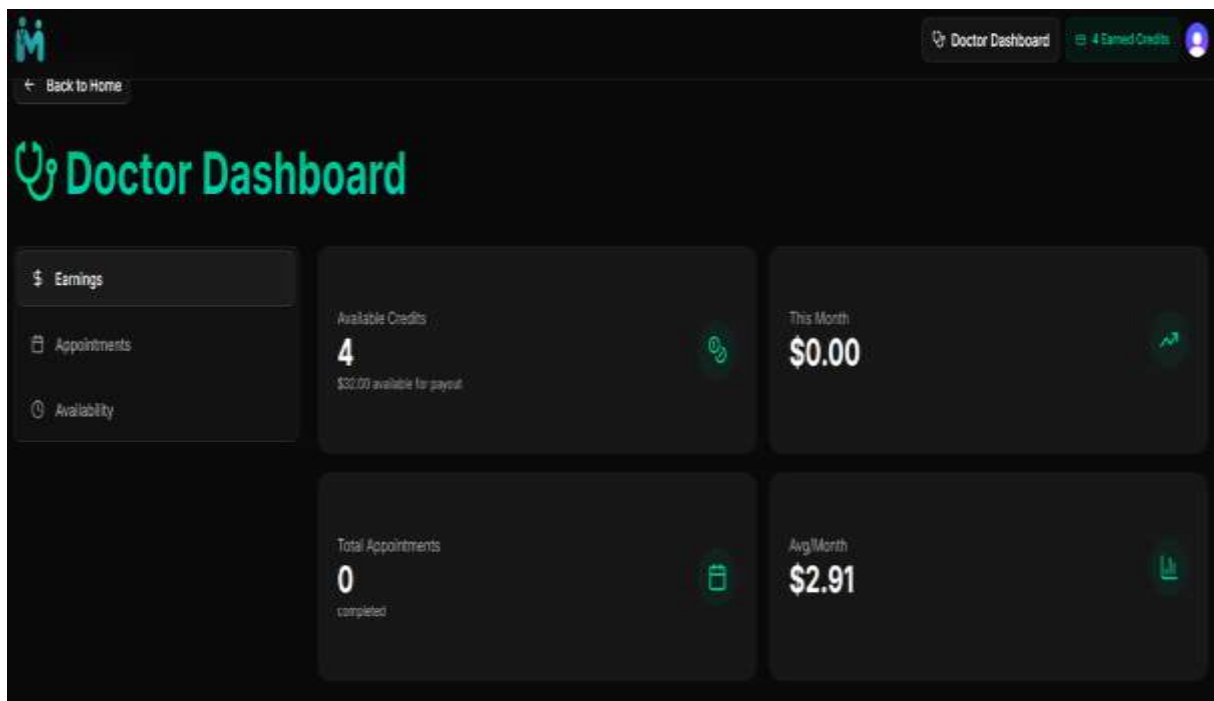


Figure 5. 4: Doctor Dashboard Screen

Figure 5.4: This screen represents the Doctor Dashboard of a healthcare platform. It provides doctors with an overview of their professional activity and account statistics in one place. The dashboard displays key metrics such as available credits, monthly earnings, total completed appointments, and average monthly income. A side menu allows doctors to navigate between sections like Earnings, Appointments, and Availability to manage their work efficiently. The interface uses a clean dark-themed layout with informational cards to highlight important data at a glance. Doctors can monitor their performance, track appointment activity, and review financial information directly from this screen. Overall, the dashboard helps doctors manage their practice and stay updated on their platform activity.

5.3.5 Admin Manage Doctors Screen

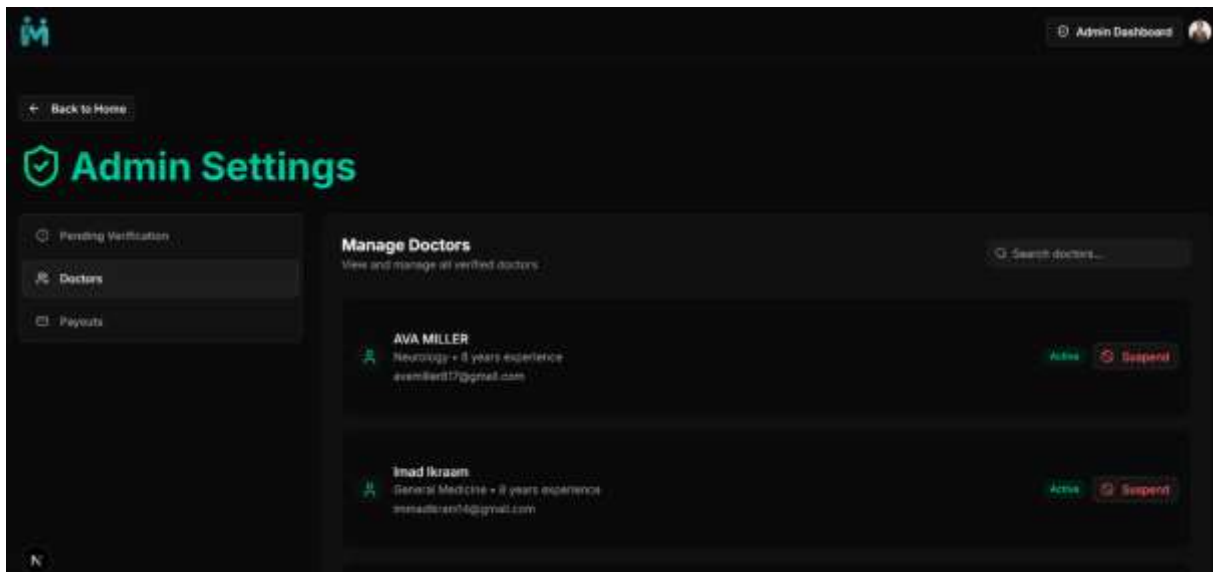


Figure 5. 5: Admin Manage Doctors Screen

Figure 5.5: This screen represents the Admin Settings – Manage Doctors section of the healthcare platform. It allows administrators to view and manage all registered and verified doctors on the system. The interface displays important details for each doctor, including their name, specialization, years of experience, and contact information. Admins can easily control the status of doctors by marking them active or suspending their accounts when necessary. A search feature is also available to quickly find specific doctors from the list.

In addition to managing doctors, the admin panel also includes options to handle pending verifications and manage payouts. Through the payout management section, administrators can review doctors' earnings and process payments based on their completed consultations and available credits. This screen is designed to give administrators full control over doctor management and financial transactions within the platform.

Chapter 6

TESTING AND EVALUATION

6 Testing and Evaluation

Testing plays a crucial role in ensuring that the MediBot telemedicine application functions correctly, meets user expectations, and provides a secure and reliable healthcare experience. Testing was performed at multiple stages of development to identify bugs, validate workflows, and ensure smooth integration between system components. Through systematic testing, the application's performance, reliability, usability, and security were continuously evaluated. In accordance with standard software engineering practices, testing was treated as a dedicated phase following implementation to allow objective assessment of system behavior. By adopting a structured testing strategy, MediBot improves software quality, enhances usability, and eliminates major issues before deployment.

6.1 Manual Testing

Manual testing was conducted to evaluate the MediBot platform's interface, navigation flow, and core functionalities. Testers interacted directly with the system to verify that each feature behaves according to the defined requirements. This testing was performed without automated tools, allowing direct observation of user experience and functionality.

During manual testing, the login and registration processes for patients, doctors, and administrators were validated. Testers confirmed correct credential handling, role-based redirection, and error messages for invalid inputs. The patient workflow was tested by searching for doctors, viewing profiles, purchasing credits, booking appointments, and joining video consultations.

Doctor-side testing included setting availability, viewing appointments, conducting video calls, and requesting payouts. The admin panel was tested for verifying doctors, managing users, and processing payouts. Edge cases such as invalid inputs, insufficient credits, overlapping time slots, and network interruptions were also tested to evaluate system robustness. Manual testing also verified responsiveness across different devices and browsers to ensure consistent user experience.

6.1.1 System Testing

System testing was performed after integrating all modules to verify overall system behavior. The testing focused on authentication, doctor search, appointment booking, video consultation, and admin control.

Table 6. 1: System Testing Table

Test ID	Feature Tested	Description	Expected Result	Actual Result	Status
ST-01	User Authentication	Test patient, doctor, and admin login/signup	Successful login/signup	As expected	Pass
ST-02	Doctor Search	Verify doctor search and filtering	Displays matching doctors	As expected	Pass
ST-03	Credit Purchase	Test credit purchase workflow	Credits updated after payment	As expected	Pass
ST-04	Appointment Booking	Verify appointment scheduling	Appointment saved and visible	As expected	Pass
ST-05	Video Consultation	Test video call connection	Video session starts successfully	As expected	Pass

Table 6.1.1.1 summarizes the system testing results. Each test case corresponds to a major system functionality such as login, booking, and video consultation. The expected results define how the system should behave, while actual results confirm real-world behavior. The pass status indicates that all major system components are functioning correctly.

6.1.2 Unit Testing

Unit testing involved testing individual modules independently to ensure each component performed correctly before integration. The MediBot platform's critical modules were tested separately.

Authentication Module tested login validation and role-based access.

Doctor Module tested availability management and profile handling.

Appointment Module tested booking creation and scheduling validation.

Credit System tested credit deduction and balance updates.

Video Call Module tested call initialization and termination.

Admin Dashboard tested verification and payout management.

Table 6. 2: Unit Function Table

Module	Function	Expected Output	Actual Output	Status
Authentication	validateUser()	Valid credentials return success	Returned success	Pass
Doctor Module	setAvailability()	Time slots saved	Saved successfully	Pass
Appointment System	createAppointment()	Appointment created	Created successfully	Pass

Credit System	updateCredits()	Credits updated	Updated correctly	Pass
Video Module	startCall()	Call session created	Created successfully	Pass

Table 6.1.2.1: provides a breakdown of unit testing results. Each function was tested independently, and outputs matched expectations. This ensured that each module worked correctly before integration.

6.1.3 Functional Testing

Functional testing ensured that all use cases defined in the system worked correctly for patients, doctors, and administrators.

Table 6. 3: Test Case Table

Test Case ID	Use Case	Input	Expected Output	Actual Output	Status
TC-01	Register / Login	Valid credentials	Redirect to dashboard	Successful login	Pass
TC-02	Find Doctor	Search by specialization	Matching doctors displayed	As expected	Pass
TC-03	Purchase Credits	Select package	Credits updated	As expected	Pass
TC-04	Book Appointment	Select doctor & time	Appointment confirmed	As expected	Pass
TC-05	Video Call	Start consultation	Video session starts	As expected	Pass

Table 6.1.3.1: presents functional test cases derived from system use cases. Each test case validates specific user interactions and confirms that the MediBot platform fulfills functional requirements.

6.1.4 Integration Testing

Integration testing ensured smooth communication between interconnected modules such as authentication, appointment system, credit system, and video consultation.

Tested Integrations:

Patient ↔ Doctor Search ↔ Appointment Booking

Credits System ↔ Appointment Booking

Doctor Availability ↔ Booking Module

Video Call ↔ Appointment Module

Admin Verification ↔ Doctor Dashboard

Table 6. 4: Integration Testing Table

Integration ID	Modules Integrated	Expected Result	Actual Result	Status	Integration ID
IN-01	Patient & Doctor Search	Matching doctors shown	As expected	Pass	IN-01
IN-02	Credits & Booking	Credits deducted on booking	As expected	Pass	IN-02
IN-03	Availability & Appointment	Only available slots booked	As expected	Pass	IN-03

Table 6.1.4.1: documents integration testing results. All modules communicated correctly and data flow between components remained consistent.

6.2 Tools and Technologies

During the testing and evaluation phases, the following tools and frameworks were employed to ensure the MediBot Platform performed efficiently and accurately. Each tool played a significant role in either developing, testing, or validating specific system functionalities. Frontend and backend frameworks ensured smooth user interaction and data processing, while AI and database technologies supported smart recommendations and reliable data handling. Testing tools like Postman and GitHub enhanced quality assurance, version control, and collaboration throughout the process:

Table 6. 5: Tools and Technologies Table

Tool / Framework	Category	Purpose / Description
React 19	Frontend Framework	Builds responsive UI for patient, doctor, and admin panels
Next.js 15	Full Stack Framework	Handles frontend rendering and backend API routes
PostgreSQL	Database	Stores users, doctors, appointments, and credits
Prisma ORM	Database ORM	Simplifies database interaction and queries
Clerk Authentication	Authentication	Secure login and role-based access
Vonage Video API	Video Integration	Real-time video consultation
Postman	API Testing Tool	Tests backend endpoints

Git & GitHub	Version Control	Code management and collaboration
Tailwind CSS	UI Styling	Responsive and modern UI design

Table 6.2.1: lists tools used in development and testing. These technologies ensured reliability, scalability, and efficient testing..

6.3 Summary

The testing and evaluation phase successfully validated the MediBot telemedicine platform's reliability, usability, and performance. Through manual, unit, functional, and integration testing, all major functionalities were verified against system requirements. The login system, doctor search, credit purchase, appointment booking, and video consultation features worked correctly.

Minor interface issues discovered during manual testing were fixed to improve usability. Integration between modules such as appointment booking, availability management, and video consultation worked seamlessly.

Overall, the testing phase confirmed that MediBot is stable, secure, and ready for deployment. The system meets project objectives by providing secure authentication, efficient appointment management, credit-based booking, and real-time video consultation within a user-friendly telemedicine platform.

Chapter 7

CONCLUSION AND FUTURE WORK



7 Conclusion

The development of the MediBot telemedicine platform successfully achieved its primary objective — to design and implement a modern, secure, and user-friendly system for connecting patients with verified medical professionals. The platform bridges the communication gap between patients, doctors, and administrators by integrating appointment management, secure video consultations, and credit-based transactions into a single intelligent healthcare ecosystem.

Throughout the project, strong emphasis was placed on usability, functionality, and scalability. The system architecture was designed using a modular approach, ensuring that each component — including authentication, appointment scheduling, credit management, video consultation, and admin controls — operates independently while integrating seamlessly with the overall system. This modular structure improved system maintainability, simplified testing, and allowed easy future enhancements.

The user interface, built using React 19, Next.js 15, and Tailwind CSS, provides an intuitive experience for patients, doctors, and administrators. The role-based dashboards ensure that each user interacts only with relevant features, improving usability and reducing complexity. The backend, powered by Prisma ORM and PostgreSQL, ensures secure and efficient handling of user data, appointment records, and transaction information. The integration of Clerk authentication enhances security, while the Vonage Video API enables real-time, reliable video consultations between patients and doctors.

Comprehensive testing — including manual testing, unit testing, functional testing, and integration testing — confirmed that all system modules performed reliably. Each feature, such as patient registration, doctor search, credit purchase, appointment booking, video consultation, and admin verification, was validated against defined requirements. The integration between appointment scheduling, credit handling, and video consultation modules proved to be stable and efficient.

In conclusion, the MediBot platform demonstrates how modern web technologies and telemedicine services can significantly improve digital healthcare accessibility. The system automates appointment scheduling, enables secure online consultations, and provides efficient workflow management for healthcare providers. MediBot fulfills its project objectives and

provides a strong foundation for future advancements in AI-assisted telemedicine and digital healthcare solutions

7.1 Future Work

While the current version of the MediBot platform fulfills its core objectives, several enhancements can further improve functionality, scalability, and real-world usability. Future improvements may include:

1. Mobile Application Development:

Developing dedicated Android and iOS mobile applications to provide easier access for patients and doctors. A mobile version would improve accessibility, allow push notifications for appointments, and enhance overall user convenience.

2. AI-Based Symptom Analysis:

Integrating AI-driven symptom checking that allows patients to enter symptoms and receive doctor recommendations. This feature would help patients identify suitable specialists before booking appointments.

4. Real-Time Chat Messaging:

Adding secure chat functionality between patients and doctors for quick consultations, follow-ups, and sharing medical information without requiring a full video call.

5. Online Payment Gateway Integration:

Integrating secure online payment gateways for purchasing credits directly using debit cards, mobile wallets, or bank transfers, making the payment process more flexible and user-friendly.

6. Prescription and Medical Records Management:

Allowing doctors to upload digital prescriptions and enabling patients to maintain medical history within the system for future consultations.

Chapter 8

REFERENCE

8 References

- [1] Vercel Inc., “Next.js Documentation,” 2024. [Online]. Available: <https://nextjs.org/docs>
- [2] Meta Open Source, “React Documentation,” 2024. [Online]. Available: <https://react.dev/learn>
- [3] Tailwind Labs, “Tailwind CSS Documentation,” 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [4] Prisma, “Prisma ORM Documentation,” 2024. [Online]. Available: <https://www.prisma.io/docs>
- [5] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [6] Clerk Inc., “Clerk Authentication Documentation,” 2024. [Online]. Available: <https://clerk.com/docs>
- [7] Vonage, “Vonage Video API Documentation,” 2024. [Online]. Available: <https://developer.vonage.com/en/video>
- [8] Axios Contributors, “Axios HTTP Client Documentation,” 2024. [Online]. Available: <https://axios-http.com/docs/intro>
- [9] Postman Inc., “Postman API Platform Documentation,” 2024. [Online]. Available: <https://learning.postman.com/>
- [10] GitHub, “GitHub Documentation,” 2024. [Online]. Available: <https://docs.github.com/>
- [11] Vercel, “Deployment and Hosting Documentation,” 2024. [Online]. Available: <https://vercel.com/docs>
- [12] Shadcn UI, “Shadcn UI Component Documentation,” 2024. [Online]. Available: <https://ui.shadcn.com/docs>
- [13] World Wide Web Consortium (W3C), “HTML & CSS Standards,” 2024. [Online]. Available: <https://www.w3.org/TR/>
- [14] UML.org, “Unified Modeling Language (UML) Specifications,” 2024. [Online]. Available: <https://www.uml.org/>
- [15] Stack Overflow, “Developer Documentation and Technical Discussions,” 2024. [Online]. Available: <https://stackoverflow.com/>